



UNIVERSIDAD DE MÁLAGA



E.T.S. INGENIERÍA
INFORMÁTICA
UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería del Software

Biblioteca extensible de optimización metaheurística en Rust

Extensible metaheuristic optimization library in Rust

Realizado por

David Muñoz del Valle

Tutorizado por

Gabriel Jesús Luque Polo

Departamento

Departamento de Lenguajes y Ciencias de la Computación

MÁLAGA, Junio 2026



UNIVERSIDAD
DE MÁLAGA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA
GRADO EN INGENIERÍA DEL SOFTWARE

Biblioteca extensible de optimización metaheurística en Rust

Extensible metaheuristic optimization library in Rust

Realizado por

David Muñoz del Valle

Tutorizado por

Gabriel Jesús Luque Polo

Departamento

Departamento de Lenguajes y Ciencias de la Computación

UNIVERSIDAD DE MÁLAGA

MÁLAGA, JUNIO 2026

Fecha defensa: Julio de 2026

Resumen

El presente Trabajo de Fin de Grado tiene como objetivo principal el diseño, desarrollo e implementación de una biblioteca de optimización metaheurística de alto rendimiento, caracterizada por una arquitectura extensible, modular y completamente autocontenida bajo una filosofía de cero dependencias externas. Para cumplir con estas directrices de diseño, el sistema ha sido programado íntegramente en Rust, un lenguaje de programación moderno que garantiza la seguridad de memoria y la ausencia de carreras de datos sin comprometer la eficiencia temporal ni el consumo de recursos de la máquina.

Con el fin de materializar este propósito principal, el proyecto se articula en torno a una serie de objetivos específicos destinados a aportar valor tanto desde el prisma de la ingeniería de software como de la computación evolutiva. En primer lugar, se persigue la definición de abstracciones sólidas mediante contratos de interfaz flexibles que desacoplen por completo la representación del dominio de los problemas de la lógica intrínseca de las metaheurísticas y sus operadores. En segundo lugar, se establece el objetivo de dar soporte nativo y transparente a escenarios de optimización tanto mono-objetivo como multiobjetivo, superando las rigideces estructurales comunes en plataformas tradicionales. Por último, la biblioteca busca proveer un motor de experimentación robusto y paralelizado, dotado de mecanismos de tolerancia a fallos y sistemas de monitorización en tiempo real, que facilite la sintonización de hiperparámetros y la realización de análisis comparativos frente a las principales herramientas de referencia de la literatura científica.

Palabras clave: Rust, Metaheurísticas, Optimización, Biblioteca

Abstract

The main objective of this Bachelor's Thesis is the design, development, and implementation of a high-performance metaheuristic optimization library, characterized by an extensible, modular, and fully self-contained architecture under a zero-external-dependencies philosophy. To comply with these design guidelines, the system has been programmed entirely in Rust, a modern programming language that guarantees memory safety and the absence of data races without compromising time efficiency or machine resource consumption.

In order to achieve this core purpose, the project is structured around a series of specific objectives aimed at providing value from the perspectives of both software engineering and evolutionary computation. First, it aims to define solid abstractions through flexible interface contracts that completely decouple the problem domain representation from the intrinsic logic of the metaheuristics and their operators. Second, it establishes the objective of providing native and transparent support for both single-objective and multi-objective optimization scenarios, overcoming the structural rigidities common in traditional frameworks. Finally, the library seeks to provide a robust and parallelized experimentation engine, equipped with fault-tolerance mechanisms and real-time monitoring systems, to facilitate hyperparameter tuning and comparative analysis against the leading reference tools in the scientific literature.

Keywords: Rust, Metaheuristics, Optimization, Library

Agradecimientos

El único apartado donde hablaré en primera persona y no porque sea en el único donde se me permite esta práctica, sino porque es el único donde realmente lo escribiré yo.

En este capítulo voy a resumir mi paso por la facultad, pero no quiero hablar sobre lo académico, más bien sobre las personas que me acompañaron.

Que viniera aquí no estaba entre mis planes, fue el sitio donde pude entrar con mi nota de selectividad, nunca tampoco tuve muy claro lo que estudiar y de rebote acabé cursando Ingeniería Informática en Málaga. Llegué solo, ningún amigo, nadie que conocía estudiaría en Málaga, sin embargo aquí, me esperaba muchísima gente increíble.

Los primeros días hice migas con los que todavía son mis amigos, un grupo maravilloso a los que les debo muchísimo, sin ellos no hubiera aguantado el primer cuatrimestre. El cuatrimestre que tan complicado se nos hizo con cálculo impartida por **Yadira** o electrónica, de la cual mejor no doy más detalles.

Aquí conocí a **Carlos**, pocas personas me hacen reír tanto. También a **Tomás**, que aunque ya no nos veamos mucho, aprendí mucho de él. Luego a **Suito** que con su amabilidad, su sonrisa y ahora con su gorra nos cautiva a todos, **Dino**, la persona más bondadosa que existe, no exagero, nunca he visto un ápice de maldad en este chico. **Sofía**, nuestra China, luego **Salma**, la persona con más imaginación que conozco y la chica con mas calle de la ETSII. La **Marta**, la señora de los viajes, se ha tirado más tiempo en aviones que estudiando, es una crack, ahora te corre 20K y se la pela. **Jose**, se describe a si mismo como un pato galáctico pero es imposible ser más listo y más gafas, te habla en 100 idiomas y siempre está para todos. Por último **Roberto**, un rondeño de ojos bonitos. La vida nos puso en el mismo gimnasio y en la misma clase.

Avanzaron los cursos, perdí el contacto con muchos y en tercero decidí cambiarme de modalidad, tuve que adaptarme a una nueva clase, a un nuevo conjunto de personas. En este cambio me acompañó Suito. Con él tenía una manía rara, nos tocábamos mucho cariñosamente las piernas. No nos quedó

otra que colarnos en un grupo para camuflarnos entre ellos y no parecer *raritos*. Este realmente no era un grupo, era un equipo, el equipo JAJER.

Con nuestra incorporación el equipo se transformó en JAJERDA, ya que el nombre se forma de las iniciales de los miembros que lo componen. La primera J, de **Joge**, una bestia de la programación, enamorado del baloncesto, la A de **Artu**, el rey del grupo, siempre con una sonrisa, el cortisol le teme a él. La siguiente J le pertenece a **Juan Manuel**. Juanma tiene varias pasiones; derrapar, los bocadillos baratos, las Ondas de Elliot, el monte, la playa y la novia de Eduardo, ... yo sinceramente soy el fan número uno de Juanma. La E obviamente, de **Eduardo**. Un cordobés pila pila fuerte, **gym rat science based**, le gusta mucho el color negro y está potente. La última letra antes de las que nos corresponden a mi y a Suito, es la R, de ni más ni menos que de **Rubén**, el G.O.A.T del equipo, la mente pensante, mi compañero del Bar **Avilés**, el friki de los periféricos, el humano en el loop (vibecodea mucho), la humildad y la generosidad personificada. Este hombre nos pasará a todos por la izquierda, en unos meses lo veremos muy alto.

Durante este año me mudé, empecé a compartir piso con Sofía, la famosa china, durante la convivencia la conocí mucho mejor y realmente me llevé una grata sorpresa, me sorprendió para bien. Sofía es increíble y realmente es capaz de lo que se proponga, tiene una capacidad de enfocarse inimaginable.

Al año siguiente, en mi último curso, me volví a mudar y cambiaron mis compañeros de piso. Estos eran Roberto, el cual ya conocéis y **Pablo**, la convivencia durante este curso fue inmejorable y estos es gracias a que ellos también lo eran. Pablo es un crack absoluto, es un futuro mayordomo de Montejaque y el fan número uno del Málaga FC. Servicial y siempre está para sus allegados. Sobre Roberto ya está todo dicho, el más cariñoso de toda ronda, le da miedo conseguir dopamina de una fuente que no sea mirar a una pared durante treinta minutos, francotirador con la honda y apasionado de la tortilla francesa y de las patatas fritas, pero al final, es el mejor.

En esta nueva edición del piso Software, se vivieron grandes incendios, grandes partidos del Albacete, intensas jornadas de repostería y escenas espirituales..... Sobre esta última debo agradecer a **Rosa**, quien me enseñó sobre la importancia de estar relajado durante estas jornadas espirituales.

Durante estos años siendo parte del equipo JAJERDA, participé en varios Hackathones, en una de estos tuve la oportunidad de conocer a un profesor del que me siempre había escuchado buenas palabras. Le pregunté sobre una duda que tenía y él estuvo ayudando de forma altruista como si el problema le

afectara de primera mano, este profesor ahora es el tutor este TFG, muchas gracias Gabriel, no eres consciente lo agradecidos que estamos los alumnos por tenerte a ti como docente.

Durante mis años en la carrera también hubieron personas que me ayudaron muchísimo, provenían de antes y siempre estuvieron allí. Obviamente hablo de mi familia y de mis amigos más cercanos. Ellos aunque suene muy cliché son de verdad los pilares de mi mismo, sin ellos yo me desplomaría, no sería capaz de nada. Con algunos he pasado todo lo que recuerdo de vida, **Agustín y Guiri**, todavía me acuerdo cuando de chicos jugábamos al fútbol y al *torito en alto* en el parque. Otros llegasteis un poco más tarde, como **Oncala**, que aunque hayamos pasado por malos momentos, es de los que más confío.

Algunos más tarde con el balonmano como **Rámirez y Sergio**, mis *compas* de Sierra Nevada, mis exhibicionistas. Otros durante el instituto, como **Barrera y Andrés**.

A diferencia de los demás casos mi familia no llegó, yo llegué a ellos, mi madre y mi padre, los que siempre están, los que creen más en mi que yo mismo, los que me daban cobijo cuando no estaba a gusto en Málaga, cuando estaba agobiado, siempre les podía llamar, cuando estaba enfermo, siempre me cuidaban y cuando quería siempre podía volver a mi casa con ellos. A ellos se lo debo todo, gracias **Papá**, gracia **Mamá**. Muchas gracias por estar. Gracias también a **Luis**, que aunque gruñón, tienes muy buen corazón, el mejor hermano que se puede tener. Gracias a **Laura** y a **Mario**, vosotros también sois mis hermanos.

Falta todavía una persona por agradecer.

Esta persona constituye un caso un poco anticlimático. Antes de los dos nacer ya eramos amigos. La vida nos mantuvo siempre cerca; en la misma clase en infantil, en la misma clase del colegio, en la misma clase del instituto, en el mismo grupo de amigos, en el mismo equipo de balonmano... Intentamos incluso irnos a estudiar a la misma ciudad para poder compartir piso y pasar juntos una etapa más.

Siempre contestaré con tu nombre cuando me pregunten sobre quien es mi mejor amigo, siempre me pusiste en primer lugar, me valoraste y me motivaste a ser mejor, a disfrutar y en los últimos momentos de tu vida, a programar.

Descansa en paz Moreno.

Resumen	1
Abstract	3
Agradecimientos	5
1 Introducción	15
1.1 Motivación	15
1.2 Objetivos	16
1.3 Tecnologías a utilizar	17
2 Metodología de trabajo	19
2.1 Modelo y ecosistema de desarrollo	19
2.2 Fases del proyecto	19
2.2.1 Investigación y capacitación	19
2.2.2 Definición de requisitos y planificación	20
2.2.3 Implementación técnica	20
2.3 Licencia de software	22
3 Optimización y Metaheurísticas	23
3.1 El Problema de Optimización	23
3.2 Métodos Exactos frente a Métodos Aproximados	24
3.3 Heurísticas y Metaheurísticas	25
3.4 Clasificación de las Metaheurísticas	26
3.4.1 Metaheurísticas basadas en trayectoria	26
3.4.2 Metaheurísticas basadas en población	26
3.5 Extensión a la Optimización Multiobjetivo	28
3.5.1 Formulación Matemática del Problema y Dominancia	28
3.5.2 Algoritmo NSGA-II (Non-dominated Sorting Genetic Algorithm II)	29
4 Especificación y diseño	31
4.1 Arquitectura	31
4.2 Módulos	32
4.2.1 Algoritmos	32
4.2.2 Experimentos	33

4.2.3	Observadores	34
4.2.4	Operadores	34
4.2.5	Problemas	35
4.2.6	Conjunto de soluciones	36
4.2.7	Soluciones	36
4.2.8	Utilidades	37
5	Implementación y pruebas	39
5.1	Implementación	39
5.1.1	Las complejas soluciones	39
5.1.2	Abstracción de Problemas y Operadores	43
5.1.3	Funcionamiento concreto de los algoritmos	46
5.1.4	Observadores y checkpoints	52
5.1.5	Experimentos	55
5.2	Desarrollo de pruebas	57
5.2.1	Pruebas unitarias	57
5.2.2	Pruebas de integración	58
6	Validación experimental	61
7	Evaluación Comparativa y Análisis de Rendimiento	65
7.1	Bibliotecas de Referencia	65
7.1.1	jMetal y jMetalPy	66
7.1.2	Pagmo (C++)	66
7.1.3	DEAP (Distributed Evolutionary Algorithms in Python)	66
7.1.4	Mealpy	66
7.2	Evaluación con la Función de Rastrigin	66
7.2.1	Resultados y Análisis	67
7.2.2	Discusión de resultados	67
7.3	Evaluación con el Problema del Viajante (TSP)	68
7.3.1	Resultados y Análisis	68
7.3.2	Interpretación de los resultados	69
8	Conclusiones y líneas futuras	71
	Apéndice A. Installation Manual	73
A.1	Requisitos	73

A.2	Instalación desde el repositorio	73
A.3	Instalación como <i>crate</i> mediante Cargo	74

1.1	Resultados de una de las encuestas a desarrolladores de Stack Overflow 2025 [18]. .	17
2.1	Captura de pantalla del estado del tablero Kanban en Jira durante la ejecución del proyecto.	21
3.1	Representación en el espacio de objetivos de un problema con dos funciones a minimizar (f_1 y f_2). La solución A domina a la solución B , ya que es estrictamente mejor en f_1 e igual o mejor en f_2	28
4.1	Arquitectura de la biblioteca. Diagrama de clases, UML. Visual Paradigm 17.0 [24]. .	32
5.1	Arquitectura de comunicación asíncrona entre hilos mediante eventos.	54
5.2	Flujo compacto de gestión de persistencia y recuperación de estados en Linux.	56
7.1	Comparativa de convergencia y tiempos de ejecución para $D = 80$	67

7.1	Resultados comparativos: Función de Rastrigin ($D = 80$, 5000 evals).	67
7.2	Resultados comparativos: TSP con Algoritmo Genético ($D = 48$, 4000 evals).	68

1

Introducción

1.1 Motivación

La optimización metaheurística constituye una herramienta fundamental para la resolución de problemas complejos en aquellos ámbitos donde los métodos exactos resultan inviables, ya sea debido al elevado coste computacional o a la naturaleza no lineal y no convexa de los espacios de búsqueda. Este tipo de técnicas ha demostrado su utilidad en una amplia variedad de aplicaciones, como la planificación y asignación de recursos, la optimización de rutas, el diseño de sistemas, la inteligencia artificial, el aprendizaje automático y la ingeniería en general [4].

En distintos lenguajes de programación existen bibliotecas maduras que facilitan la utilización de técnicas metaheurísticas. Por ejemplo, *jMetal* es un framework desarrollado por investigadores de la Universidad de Málaga, ampliamente reconocido para la optimización multiobjetivo con metaheurísticas [11]. Su versión en Python, *jMetalPy*, ofrece un entorno extensible y orientado a objetos para experimentar con técnicas clásicas y avanzadas de optimización [7]. Asimismo, *MEALPY* (MEtaheuristic ALgorithms in PYthon) proporciona implementaciones de numerosos algoritmos inspirados en la naturaleza, tanto clásicos como híbridos, para abordar problemas complejos [25]. Sin embargo, dentro del ecosistema de Rust [21], el soporte para la optimización metaheurística es todavía limitado, lo que dificulta la adopción de estas metodologías en proyectos desarrollados en dicho lenguaje y pone de manifiesto la necesidad de disponer de herramientas nativas que se integren de forma natural con sus principios de diseño, especialmente en lo relativo al rendimiento, la seguridad de memoria y el control de recursos.

Este Trabajo de Fin de Grado surge con la motivación de dar respuesta a esta necesidad mediante el desarrollo de una biblioteca de optimización metaheurística modular y extensible. No obstante, abordar un proyecto de esta envergadura resulta especialmente desafiante en las etapas iniciales,

particularmente en ausencia de experiencia previa tanto en las técnicas de optimización como en el lenguaje de programación Rust. Afrontar un grado de fin de carrera implica recopilar y consolidar los conocimientos adquiridos a lo largo de la titulación y aplicarlos en un proyecto real, lo cual supone una tarea de notable complejidad. Por ello, la elección de un tema que despierte interés, curiosidad y motivación personal resulta un factor clave para el desarrollo del proyecto.

El diseño y desarrollo de algoritmos metaheurísticos conforma un campo especialmente atractivo, al combinar fundamentos de la teoría de la computación, la optimización y la inteligencia artificial. Su aplicación en dominios tan diversos como la logística, la ingeniería o la biología computacional pone de manifiesto su versatilidad y su impacto en la resolución de problemas reales.

Como se ha comentado, en el momento de inicio de este proyecto, no existían bibliotecas consolidadas orientadas al desarrollo de este tipo de soluciones en el lenguaje Rust, lo que constituyó una motivación adicional para su realización. Asimismo, el propio lenguaje Rust actuó como detonante del proyecto, al tratarse de una tecnología moderna y en auge, diseñada con un fuerte énfasis en la seguridad de memoria y el rendimiento, y especialmente adecuada tanto para el aprendizaje como para el desarrollo de software robusto y eficiente.

1.2 Objetivos

El objetivo principal de este proyecto es desarrollar una biblioteca en Rust que implemente algoritmos metaheurísticos para la optimización de problemas complejos. Debe ser capaz de resolver correctamente gran variedad de problemas de único objetivo y un conjunto de problemas multiobjetivo.

Idealmente, esta biblioteca debería ser fácil de usar, eficiente y extensible, permitiendo a los usuarios adaptar los algoritmos a sus necesidades específicas.

Con el propósito de demostrar el potencial, la viabilidad y el rendimiento de la herramienta, se implementará un conjunto seleccionado de problemas y algoritmos de referencia que servirán como base de pruebas. Sobre esta base, se llevarán a cabo validaciones experimentales y comparaciones exhaustivas, analizando críticamente los resultados obtenidos para evaluar el comportamiento real y la eficiencia de la biblioteca.

Para garantizar el cumplimiento de estas premisas, se realizará un desglose exhaustivo de los requisitos del sistema, donde se definirán formalmente las funcionalidades y restricciones que guiarán el desarrollo de la herramienta.

1.3 Tecnologías a utilizar

Como lenguaje de programación, se ha elegido **Rust**, debido a su enfoque en el rendimiento, por su capacidad de manejo eficiente de la concurrencia y diferentes hilos, lo que lo hace ideal para la implementación de algoritmos metaheurísticos complejos que a menudo requieren un procesamiento paralelo intensivo. Además, su creciente ecosistema y su tipo de sistema robusto nos permiten construir soluciones de optimización escalables y de alto rendimiento. También, Rust ha comenzado a consolidarse como uno de los lenguajes de programación utilizados en el desarrollo del *kernel* de Linux, dejando atrás su carácter experimental y siendo aprobado oficialmente para la implementación de determinados componentes del núcleo, lo que refuerza su madurez, fiabilidad y adecuación para sistemas críticos [15].

Uno de los principales motivos por el que Rust es tan buena elección es por su gestor de paquetes y compilación, **Cargo**, que facilita la gestión de dependencias y la construcción de proyectos, permitiendo a los desarrolladores centrarse en la implementación de algoritmos en lugar de en la configuración del entorno.

Este gestor de paquetes se encuentra entre los más populares y admirados por los desarrolladores, como se puede observar en la estadística de la encuesta anual de Stack Overflow, donde Cargo destaca como uno de los gestores de paquetes más valorados para el desarrollo e infraestructura en la nube durante 2025 [18].

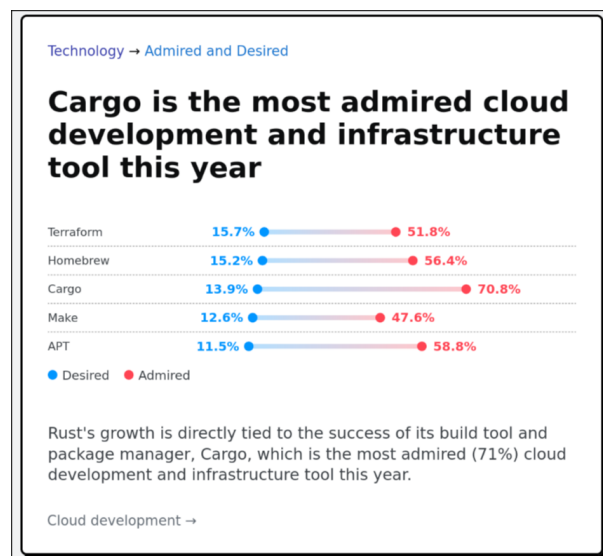


Figura 1.1. Resultados de una de las encuestas a desarrolladores de Stack Overflow 2025 [18].

En cuanto a las herramientas de desarrollo, se utilizó **Visual Studio Code** [17] como entorno de desarrollo integrado (IDE) debido a su flexibilidad, amplia gama de extensiones y soporte para Rust a

través de la extensión *rust-analyzer* [2]. Durante el desarrollo más temprano, también se usó **Rust Rover** [13], de **JetBrains** [12], que es un IDE específico para Rust. Esto fue debido a que en ese momento los conocimientos sobre Rust eran muy limitados y se necesitaba un entorno más guiado e invasivo para poder progresar.

Para la gestión del proyecto se ha utilizado **Jira** [1]. La documentación y redacción del proyecto se han realizado mediante $\text{\LaTeX}$ [20]. Por otro lado, el control de versiones se ha llevado a cabo utilizando **Git** [22], empleando **GitHub** [9] como plataforma de alojamiento remoto del repositorio.

La gestión y planificación temporal de las tareas se ha articulado mediante **Jira** [1], mientras que la composición y redacción de la memoria técnica se han elaborado en $\text{\LaTeX}$ [20]. En lo relativo a la ingeniería de software, el control de versiones se ha estructurado en torno al sistema descentralizado **Git** [22], empleando la plataforma **GitHub** [9] para el alojamiento del repositorio remoto.

La adopción de este ecosistema responde a la necesidad de garantizar la integridad y la trazabilidad del código fuente durante el ciclo de vida del desarrollo. Por un lado, la naturaleza distribuida de Git permite la experimentación segura y el aislamiento de nuevos algoritmos sin comprometer la estabilidad de la rama principal de la biblioteca. Por otro lado, GitHub aporta una infraestructura robusta para la persistencia de los datos, optimizando el seguimiento de las líneas de desarrollo e introduciendo un marco idóneo para la futura automatización de pruebas.

2

Metodología de trabajo

En esta sección se describen los procedimientos, herramientas y fases seguidas para la consecución de los objetivos del proyecto.

2.1 Modelo y ecosistema de desarrollo

Para el desarrollo del proyecto se ha adoptado una metodología iterativa, inspirada en el marco de trabajo SCRUM, pero adaptada a las particularidades de un entorno de trabajo unipersonal. Esta adaptación permite conservar las ventajas de la planificación incremental y la revisión continua, ajustándolas a la naturaleza y alcance del proyecto.

Como herramienta de apoyo a la gestión y seguimiento de las tareas, se ha utilizado JIRA [1], cuyo uso y configuración dentro del proyecto se detallarán con mayor profundidad en el apartado siguiente.

Por otra parte, las herramientas de desarrollo empleadas en este proyecto corresponden a las descritas conceptualmente en la **Sección 1.3**. La selección de este ecosistema técnico se orientó a maximizar la eficiencia en la detección de errores y la navegación por el código fuente, consolidando a Visual Studio Code como el entorno de desarrollo principal debido a su versatilidad y al robusto soporte proporcionado por la extensión *rust-analyzer* [2].

2.2 Fases del proyecto

Un proyecto como el que nos encontramos debe estructurarse por partes, en este caso en tres etapas diferenciadas:

2.2.1 Investigación y capacitación

Debido a la pronunciada curva de aprendizaje de Rust, esta fase inicial se centró en la asimilación de sus fundamentos sintácticos y las particularidades de su compilador. Para consolidar estos conceptos, se llevó a cabo la migración de un proyecto personal previo (una aplicación de escritorio), trasladando

su arquitectura basada en Java y Javalin hacia un ecosistema nativo en Rust mediante el framework Tauri [23].

Paralelamente, se desarrolló un repositorio de pruebas a modo de entorno de experimentación, destinado a la implementación de pequeños módulos funcionales que permitieron dominar aún más el lenguaje. Esta etapa de formación técnica se complementó con un estudio exhaustivo sobre metaheurísticas, especialmente, sobre algoritmos genéticos, sentando las bases teóricas necesarias para el posterior diseño de la biblioteca.

Dentro del estudio técnico, cobró especial relevancia el análisis del repositorio y de la arquitectura de jMetal [11], un framework de optimización metaheurística multiobjetivo escrita en Java por investigadores de la Universidad de Málaga, ayudó a comprender los patrones de diseño y las estructuras de datos necesarias para implementar metaheurísticas de forma extensible.

2.2.2 Definición de requisitos y planificación

Durante esta etapa se realizó un trabajo de introspección, se analizaron los objetivos y se elaboró un documento donde se especificaron todos los requisitos del sistema.

A partir de estos requisitos, se creó un *backlog* en Jira, donde se registraron todas las tareas necesarias para completar el proyecto. Posteriormente, las tareas se organizaron en seis *sprints* y se clasificaron en las columnas «Por hacer», «En curso», «Por revisar» y «Listo». Esta organización permitió un seguimiento claro del progreso y facilitó la comunicación con el tutor, mostrando periódicamente las tareas en la columna «Por revisar» para recibir su aprobación. Una vez que el tutor daba el visto bueno, las tareas se marcaban como «Listo», asegurando así un flujo de trabajo controlado y transparente [1]. En la Figura 2.1 se ilustra una captura de pantalla del estado del tablero de gestión durante el ciclo de vida del desarrollo, reflejando la distribución real de las tareas en sus respectivas fases.

2.2.3 Implementación técnica

Una vez definidos los requisitos del sistema y establecida la planificación del proyecto, se procedió a la implementación técnica de la biblioteca de optimización metaheurística. Esta fase tuvo como objetivo materializar las decisiones de diseño adoptadas previamente, dando lugar a un sistema funcional, modular y extensible, desarrollado íntegramente en el lenguaje Rust.

El proceso de implementación se llevó a cabo de forma incremental, permitiendo validar progresivamente las abstracciones definidas y adaptar la arquitectura a las particularidades del lenguaje y a los requisitos detectados durante el desarrollo.

La implementación estuvo fuertemente condicionada por las características propias del lenguaje

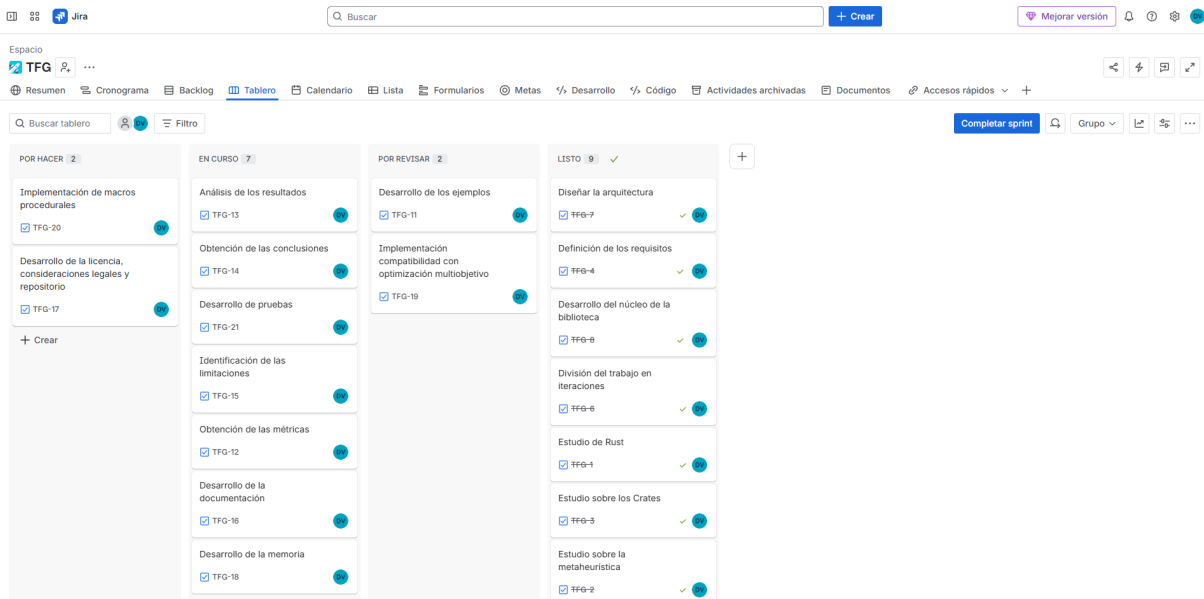


Figura 2.1. Captura de pantalla del estado del tablero Kanban en Jira durante la ejecución del proyecto.

Rust. La ausencia de herencia clásica y el uso intensivo de *traits* influyeron directamente en el diseño de las abstracciones, fomentando un enfoque basado en la composición y el polimorfismo estático.

Asimismo, el sistema de propiedad y préstamos de Rust actuó como una herramienta de validación temprana, obligando a definir de forma explícita la gestión de memoria y el ciclo de vida de las estructuras de datos. Aunque este modelo supuso una dificultad inicial, permitió garantizar un alto nivel de seguridad y robustez en la implementación final.

La implementación de la biblioteca se realizó siguiendo un orden lógico que permitió construir el sistema de forma progresiva. En primer lugar, se desarrollaron las abstracciones básicas, tales como las representaciones de soluciones y problemas. Posteriormente, se implementaron los algoritmos de optimización, seguidos de los operadores fundamentales y de los mecanismos de observación.

Este enfoque permitió validar cada componente de manera aislada antes de integrarlo en el sistema global, reduciendo la complejidad del proceso de depuración y facilitando la detección de errores conceptuales en fases tempranas del desarrollo.

Durante la fase de implementación se llevó a cabo una validación continua del sistema, apoyándose principalmente en el compilador de Rust como herramienta de detección de errores y en la ejecución de pruebas y ejemplos controlados para verificar el comportamiento de los algoritmos.

Asimismo, se realizaron refactorizaciones periódicas del código con el objetivo de mejorar su claridad, reducir duplicidades y adaptar la arquitectura a nuevas necesidades detectadas durante el desarrollo. Este proceso iterativo permitió alcanzar una implementación estable y coherente con los objetivos iniciales del proyecto.

2.3 Licencia de software

Con el propósito de fomentar la transferencia tecnológica, garantizar la transparencia del código y contribuir activamente al ecosistema de software libre y de código abierto (*open-source*), este proyecto se distribuye bajo los términos de la **Licencia MIT** [16]. La elección de este marco legal responde a una decisión estratégica alineada con las prácticas habituales del entorno de desarrollo en Rust, donde se prioriza la máxima reutilización del software con las menores restricciones posibles.

La Licencia MIT se caracteriza por ser una licencia de tipo permisivo. A diferencia de las licencias con *copyleft* fuerte (como la familia GPL), el modelo MIT concede a cualquier persona que obtenga una copia del software el derecho perpetuo, gratuito e irrevocable de utilizar, modificar, fusionar, publicar, distribuir, sublicenciar y comercializar la biblioteca sin necesidad de liberar las modificaciones bajo las mismas condiciones. Las únicas obligaciones impuestas por este marco legal se reducen a dos condiciones fundamentales:

- **Preservación del aviso de copyright:** Cualquier distribución del software, ya sea en su forma original o modificada, debe incluir obligatoriamente el aviso de derechos de autor y el texto de la licencia original.
- **Exención de responsabilidad:** Se estipula explícitamente que el software se proporciona "tal cual", eximiendo a los autores de cualquier reclamación, daño o responsabilidad legal derivada de su uso, ejecución o mal funcionamiento.

Esta permisividad resulta fundamental, ya que elimina barreras legales y reduce la fricción burocrática asociada a su adopción. Gracias a ello, tanto investigadores del ámbito académico como desarrolladores del sector industrial pueden integrar, auditar y adaptar el motor de optimización en sistemas propietarios o comerciales sin comprometer su propiedad intelectual. Esto favorece la transferencia tecnológica, amplía el alcance de la solución y refuerza la sostenibilidad e impacto del proyecto a largo plazo.

3

Optimización y Metaheurísticas

En numerosas disciplinas de la ingeniería, la economía y las ciencias de la computación, surge de manera recurrente la necesidad de tomar decisiones que maximicen el beneficio o minimicen el coste de un sistema dado. Este proceso de búsqueda de la mejor alternativa posible, dentro de un conjunto de soluciones factibles, es lo que se conoce formalmente como optimización. En este capítulo se introducen los conceptos fundamentales de la optimización matemática, las limitaciones de los métodos clásicos y el papel crucial que desempeñan las metaheurísticas en la resolución de problemas complejos.

3.1 El Problema de Optimización

De forma general, la optimización matemática trata de encontrar la mejor solución posible, evaluada según un criterio cuantitativo, de entre un conjunto de alternativas disponibles. Formalmente, un problema de optimización mono-objetivo se formula como la búsqueda de un vector de variables de decisión $x = (x_1, x_2, \dots, x_n)$ que optimice (ya sea minimizando o maximizando) una función objetivo escalar $f(x)$. Este problema se define matemáticamente de la siguiente manera:

Minimizar $f(x)$

sujeto a $x \in \Omega$

Sin pérdida de generalidad, en la literatura algorítmica suele trabajarse exclusivamente con problemas de minimización, dado que cualquier problema de maximización puede transformarse trivialmente multiplicando la función objetivo por -1 (es decir, $\max f(x) \equiv \min -f(x)$).

El vector x representa las incógnitas del sistema sobre las cuales el proceso de resolución tiene control directo. La función objetivo $f : \Omega \rightarrow \mathbb{R}$ es la métrica que asigna un valor de calidad o coste a cada vector propuesto.

El conjunto Ω denota el espacio de búsqueda o la región factible del problema. Lejos de ser un espacio infinito o arbitrario, en los escenarios de ingeniería y del mundo real, este espacio está delimitado geométrica y matemáticamente por un conjunto de restricciones que el vector x debe cumplir de manera estricta. Estas restricciones imponen límites físicos, lógicos o económicos al sistema, y suelen expresarse mediante funciones de desigualdad ($g_i(x) \leq 0, i = 1, \dots, m$) y de igualdad ($h_j(x) = 0, j = 1, \dots, p$).

Por consiguiente, Ω está constituido única y exclusivamente por aquellos vectores que satisfacen simultáneamente todas estas condiciones. Si un algoritmo de optimización genera una solución que recae fuera de las fronteras de Ω , dicha solución se considera infactible. En el diseño de algoritmos y metaheurísticas, las soluciones infactibles suponen un desafío; por lo general, son descartadas inmediatamente o, de forma más sofisticada, se les aplica una penalización severa en su valor de $f(x)$ para guiar la búsqueda iterativa de vuelta hacia el interior de la región factible.

Por consiguiente, Ω está constituido única y exclusivamente por aquellos vectores que satisfacen simultáneamente todas estas condiciones. Si un algoritmo de optimización genera una solución que recae fuera de las fronteras de Ω , dicha solución se considera infactible. En el diseño de algoritmos y metaheurísticas, las soluciones infactibles suponen un desafío; por lo general, son descartadas inmediatamente, aunque se les puede aplicar de forma más sofisticada una penalización severa en su valor de $f(x)$ para guiar la búsqueda iterativa de vuelta hacia el interior de la región factible o en tercer lugar, se puede intentar su reparación mediante un proceso normalmente determinista pero subóptimo que las transforma en soluciones factibles.

3.2 Métodos Exactos frente a Métodos Aproximados

Para abordar la resolución de los problemas matemáticos de optimización, existen tradicionalmente dos grandes familias de algoritmos: los métodos exactos y los métodos aproximados. La elección entre un enfoque u otro depende intrínsecamente de la naturaleza del problema, el tamaño de la instancia a resolver y los recursos computacionales disponibles.

Los métodos exactos exploran el espacio de búsqueda de una manera sistemática y exhaustiva, de tal forma que garantizan matemáticamente encontrar la solución óptima global del problema. Entre las técnicas más destacadas de esta familia se encuentran la Programación Lineal (como el algoritmo Simplex), la Programación Dinámica y los algoritmos de Ramificación y Acotación (*Branch and Bound*).

Cuando el problema presenta un modelo matemático tratable y un tamaño de entrada moderado, estos métodos son, sin lugar a duda, la opción preferida debido a la certeza de sus resultados.

Sin embargo, su aplicabilidad se ve severamente limitada en la práctica industrial y de investigación [19]. Muchos de los problemas de mayor interés y aplicabilidad en el mundo real pertenecen a las clases de complejidad NP-completos o NP-duros. Un ejemplo paradigmático es el Problema del Viajante de Comercio (TSP, por sus siglas en inglés) [19], donde se busca la ruta más corta que visite un conjunto de ciudades. En estos escenarios, el espacio de soluciones posibles crece de forma factorial o exponencial con respecto al tamaño del problema ($O(n!)$ o $O(2^n)$). Este fenómeno, conocido como "explosión combinatoria" o "la maldición de la dimensionalidad", provoca que un algoritmo exacto requiera tiempos de ejecución inasumibles (años o incluso siglos de cómputo) para instancias de tamaño realista [14].

Es precisamente ante esta intratabilidad computacional donde los métodos aproximados, concretamente los algoritmos heurísticos, cobran un protagonismo absoluto y se convierten en la única alternativa viable. La principal motivación de un heurístico es encontrar una solución en un tiempo razonable, especialmente para problemas NP-completos o de gran escala, donde un algoritmo exacto tardaría demasiado o sería inviable. Estos algoritmos logran esta eficiencia sacrificando la garantía de encontrar la solución óptima o perfecta.

A diferencia de los enfoques exactos, la naturaleza de los métodos aproximados es inherentemente incompleta, ya que no exploran todo el espacio de soluciones posibles. En su lugar, producen soluciones que son buenas o cercanas al óptimo (también conocidas como soluciones cuasi-óptimas), pero no tienen pruebas matemáticas que aseguren que la solución encontrada sea la mejor de todas.

3.3 Heurísticas y Metaheurísticas

El término *heurística* (del griego *heuriskein*, "encontrar" o "descubrir") hace referencia a estrategias que utilizan conocimiento específico del dominio del problema o "reglas prácticas" para guiar la búsqueda hacia soluciones de alta calidad. Si bien producen soluciones cuasi-óptimas de forma rápida, a menudo corren el riesgo de quedar atrapadas en óptimos locales (soluciones que son las mejores en su vecindario inmediato, pero no en todo el espacio de búsqueda).

Para superar esta limitación, nacen las **metaheurísticas**. Una metaheurística se define como un marco algorítmico de nivel superior, independiente del problema subyacente, que provee un conjunto de directrices para guiar a otras heurísticas subordinadas en la exploración del espacio de soluciones.

El éxito de cualquier metaheurística reside en mantener un delicado equilibrio dinámico entre dos conceptos fundamentales:

- **Exploración (o diversificación):** La capacidad del algoritmo para investigar regiones globales y desconocidas del espacio de búsqueda, evitando la convergencia prematura.
- **Explotación (o intensificación):** La capacidad de concentrar la búsqueda en las vecindades de las mejores soluciones encontradas hasta el momento, con el fin de refinar y mejorar dichos resultados [4].

3.4 Clasificación de las Metaheurísticas

A lo largo de las últimas décadas, la literatura científica ha propuesto numerosas metaheurísticas, muchas de ellas inspiradas en procesos naturales, biológicos o físicos. La clasificación más extendida en la comunidad académica divide estos algoritmos según el número de soluciones con las que trabajan simultáneamente en cada iteración:

3.4.1 Metaheurísticas basadas en trayectoria

También conocidas como métodos de búsqueda puntual, estos algoritmos mantienen y modifican una única solución a lo largo del tiempo, describiendo una trayectoria a través del espacio de búsqueda. En cada iteración, la solución actual se reemplaza por otra proveniente de su vecindario si se cumple un determinado criterio de aceptación. Ejemplos clásicos de esta categoría son la Búsqueda Local, el Recocido Simulado (*Simulated Annealing*) y la Búsqueda Tabú.

3.4.2 Metaheurísticas basadas en población

A diferencia de los métodos de trayectoria, estas técnicas mantienen en memoria un conjunto (o población) de soluciones que evolucionan de manera simultánea en cada iteración. Esta característica les confiere una mayor robustez y una capacidad de exploración superior, ya que las soluciones pueden interactuar e intercambiar información entre sí. Dentro de este paradigma destacan los Algoritmos Evolutivos (como los Algoritmos Genéticos) y la Optimización por Enjambre de Partículas (PSO).

Los Algoritmos Genéticos, fuertemente fundamentados en la teoría de la evolución biológica y la genética de poblaciones, abordan la búsqueda iterando sobre un conjunto de soluciones codificadas como individuos o cromosomas. El progreso hacia la región factible y óptima del espacio de búsqueda se dirige mediante tres mecanismos fundamentales aplicados de forma secuencial. Inicialmente, un mecanismo de selección evalúa la función objetivo de la población y discrimina probabilísticamente a los individuos más aptos, asegurando la supervivencia del mejor material genético. A continuación, el operador de cruce fomenta la explotación del espacio mediante el intercambio de segmentos del vector de variables

entre pares de individuos seleccionados, generando una descendencia que hereda características de sus progenitores. Finalmente, un operador de mutación introduce perturbaciones de carácter aleatorio en la configuración genética de la descendencia; esta inyección de variabilidad constituye la principal herramienta de exploración del algoritmo, mitigando el riesgo de convergencia prematura en mínimos locales. Tras la aplicación de estos operadores, se ejecuta una política de reemplazo que define la transición hacia la siguiente generación, incorporando frecuentemente mecanismos de elitismo para retener incondicionalmente la mejor solución descubierta.

Por su parte, la Optimización por Enjambre de Partículas (PSO) modela una dinámica sustancialmente distinta, inspirada en el comportamiento colectivo exhibido por agregaciones biológicas, tales como el vuelo sincronizado de bandadas de aves o el nado de bancos de peces. En este modelo, la población se conceptualiza como un enjambre y las soluciones candidatas actúan como partículas que se desplazan de manera continua y autónoma a lo largo del hiperespacio de búsqueda de dimensión n . A diferencia de los modelos evolutivos, en PSO no existe la destrucción ni creación de nuevos individuos, sino la perturbación guiada de sus trayectorias.

Cada partícula se encuentra definida unívocamente en una iteración t por su posición o vector de variables $x_i(t)$ y por su vector de velocidad $v_i(t)$. La cinemática del enjambre se articula mediante la actualización iterativa de estas magnitudes vectoriales. La nueva velocidad de una partícula se calcula como una combinación lineal de tres factores principales: un factor de inercia que perpetúa el momento de la trayectoria previa fomentando la exploración global; un componente cognitivo que dirige a la partícula hacia la mejor posición histórica que ella misma ha experimentado, denominada $pbest$; y un componente social que atrae a la partícula hacia el óptimo global descubierto por la totalidad del enjambre hasta ese momento, identificado como $gbest$.

Matemáticamente, la modificación del vector velocidad se expresa de la forma:

$$v_i(t+1) = w \cdot v_i(t) + c_1 \cdot r_1 \cdot (pbest_i - x_i(t)) + c_2 \cdot r_2 \cdot (gbest - x_i(t)) \quad (3.1)$$

Donde w denota el coeficiente de inercia, c_1 y c_2 representan las constantes de aceleración cognitiva y social respectivamente, y r_1, r_2 son variables aleatorias extraídas de una distribución uniforme en el intervalo $[0, 1]$ que aportan el necesario grado de estocasticidad al sistema. Inmediatamente después, el vector de posición se actualiza integrando algebraicamente la nueva velocidad:

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad (3.2)$$

Este esquema de actualización garantiza un equilibrio dinámico entre la autonomía exploratoria de cada solución y la convergencia acelerada inducida por el conocimiento topológico compartido por toda

la red de la población.

3.5 Extensión a la Optimización Multiobjetivo

En gran parte de las aplicaciones del mundo real, los problemas no presentan un único objetivo a optimizar, sino múltiples objetivos que a menudo se encuentran en conflicto directo (por ejemplo, maximizar el rendimiento de un componente a la par que se minimiza su coste de fabricación).

En estos escenarios, denominados Problemas de Optimización Multiobjetivo (MOP), el concepto de «solución óptima» desaparece para dar paso al concepto de *Dominancia de Pareto*. El objetivo de las metaheurísticas multiobjetivo no es encontrar un único punto, sino aproximar el Frente de Pareto: un conjunto de soluciones no dominadas que representan los mejores compromisos posibles entre los diferentes objetivos evaluados, permitiendo finalmente al tomador de decisiones elegir la alternativa que mejor se adapte a sus necesidades.

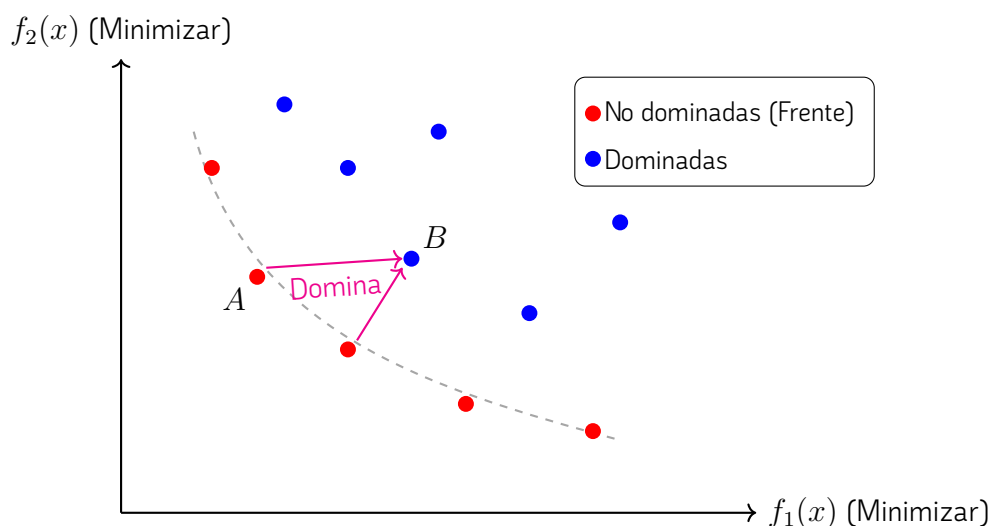


Figura 3.1. Representación en el espacio de objetivos de un problema con dos funciones a minimizar (f_1 y f_2). La solución A domina a la solución B , ya que es estrictamente mejor en f_1 e igual o mejor en f_2 .

3.5.1 Formulación Matemática del Problema y Dominancia

Matemáticamente, un Problema de Optimización Multiobjetivo (MOP) sin restricciones se define como la minimización simultánea de un vector de M funciones objetivo, expresado de la forma $\min \vec{f}(\vec{x}) = [f_1(\vec{x}), f_2(\vec{x}), \dots, f_M(\vec{x})]^T$, donde $\vec{x} \in \Omega$ es el vector de variables de decisión dentro del espacio de búsqueda factible. En este contexto analítico, se establece que una solución $\vec{x}_1$ domina a otra solución $\vec{x}_2$, lo cual se denota formalmente como $\vec{x}_1 \prec \vec{x}_2$, si y solo si $\vec{x}_1$ no es peor que $\vec{x}_2$

en ninguno de los objetivos, es decir, $\forall i \in \{1, \dots, M\}, f_i(\vec{x}_1) \leq f_i(\vec{x}_2)$, y además es estrictamente superior en al menos un objetivo, lo que implica que $\exists j \in \{1, \dots, M\} \mid f_j(\vec{x}_1) < f_j(\vec{x}_2)$.

3.5.2 Algoritmo NSGA-II (Non-dominated Sorting Genetic Algorithm II)

Para resolver este tipo de escenarios, uno de los algoritmos más extendidos e influyentes en la literatura científica es el NSGA-II, propuesto por K. Deb et al [5]. Este algoritmo genético basa su eficacia en dos pilares matemáticos fundamentales que guían la evolución de la población: un mecanismo de ordenación rápida no dominada (Fast Non-Dominated Sort) y un estimador de la densidad del entorno denominado distancia de apilamiento (Crowding Distance).

En primer lugar, la población combinada R_t de tamaño $2N$ (formada por la unión de la población de padres P_t y su descendencia Q_t) se evalúa y clasifica en diferentes frentes de Pareto $\mathcal{F} = \{F_1, F_2, \dots, F_k\}$. El primer frente F_1 contiene exclusivamente aquellas soluciones que no son dominadas por ninguna otra dentro del conjunto R_t . El frente F_2 contiene a las soluciones dominadas únicamente por los individuos presentes en F_1 , y el proceso continúa iterativamente. A cada solución i se le asigna un rango o nivel de no dominancia r_i equivalente al índice de su frente, de modo que un rango menor indica una calidad global superior de la solución.

En segundo lugar, para garantizar la diversidad algorítmica y promover una distribución uniforme de los individuos a lo largo del Frente de Pareto, se calcula la distancia de apilamiento d_i para cada individuo exclusivamente respecto a las soluciones de su mismo frente. Para un objetivo m específico, se ordenan los individuos del frente en función de su valor en dicho objetivo. A las soluciones límite (aquellas que determinan los extremos del frente) se les asigna una distancia matemática infinita ($d_1 = d_l = \infty$), garantizando su preservación. Para el resto de soluciones intermedias i , la distancia se computa acumulando las diferencias normalizadas entre sus vecinos topológicos adyacentes:

$$d_i = \sum_{m=1}^M \frac{f_m^{(i+1)} - f_m^{(i-1)}}{f_m^{\max} - f_m^{\min}} \quad (3.3)$$

donde $f_m^{(i+1)}$ y $f_m^{(i-1)}$ son los valores del objetivo m para los individuos vecinos de la solución i , mientras que $f_m^{\max}$ y $f_m^{\min}$ representan los valores máximo y mínimo empíricos registrados para ese objetivo concreto dentro del frente en evaluación.

Finalmente, la convergencia del método se garantiza utilizando el operador de comparación atestada o *crowded-comparison operator* ($\prec_n$) para seleccionar determinísticamente a los N mejores individuos que conformarán la población de la siguiente generación P_{t+1} . Dadas dos soluciones i y j , el algoritmo establece que $i \prec_n j$ (y por tanto i prevalece) si el rango de dominancia de i es estrictamente mejor

que el de j ($r_i < r_j$), o bien, en caso de empate al pertenecer ambos al mismo frente ($r_i = r_j$), si la solución i reside en una región del espacio de objetivos menos poblada que j , maximizando su distancia de apilamiento ($d_i > d_j$).

4

Especificación y diseño

4.1 Arquitectura

El diseño y modelado de la arquitectura representan la fase de mayor complejidad y dedicación en el desarrollo. La integridad de esta estructura determina la calidad técnica, la eficiencia y la mantenibilidad de la biblioteca. Con el objetivo de garantizar un nivel de abstracción óptimo que permita su aplicación en diversos casos de uso, se han integrado los siguientes criterios de diseño;

La arquitectura desacopla la definición del problema de la lógica del optimizador. Los límites del espacio de búsqueda y las restricciones se encapsulan en una estructura específica. Esta entidad es referenciada por el motor de ejecución, permitiendo que el algoritmo identifique el dominio de las variables. De este modo, el algoritmo puede realizar las evaluaciones e iteraciones pertinentes respetando estrictamente las directrices del sistema a optimizar.

Los algoritmos se han diseñado de forma modular, fundamentándose en el uso de operadores genéricos que se configuran durante la fase de instanciación. Estos operadores son los responsables de la transformación y evolución de las soluciones. Se distinguen tres categorías principales, operadores de mutación, de cruce y de selección.

Todas las implementaciones de los algoritmos tienen varias estructuras asociadas. Estas son los **parámetros**, que definen el comportamiento del algoritmo, el **estado de ejecución**, que permite la trazabilidad de cada paso, dado que el sistema emplea un modelo de ejecución secuencial por etapas y luego el **conjunto de soluciones**, que representa el resultado final.

Un componente diferenciador es su sistema de experimentación integrado. A partir de la parametrización de los algoritmos, la biblioteca facilita la ejecución comparativa de múltiples configuraciones sobre un mismo problema. Este módulo automatiza la obtención de métricas, tales como tiempos de ejecución y calidad de la solución obtenida, facilitando así la elección del algoritmo más adecuado para solventar

un problema específico.

El estado de ejecución actúa como el mecanismo principal para actualizar el contexto del algoritmo, permitiendo verificar de forma continua si se han cumplido los criterios de terminación definidos en la parametrización inicial. Este contexto no solo regula el flujo de control del proceso, sino que sirve como intermediario crítico para la comunicación con los observadores del sistema. Mediante el uso de hilos de ejecución independientes, el contexto transmite los datos de la evolución del algoritmo en tiempo real, facilitando tanto la monitorización por parte del usuario como el almacenamiento persistente de métricas para la generación posterior de reportes de rendimiento.

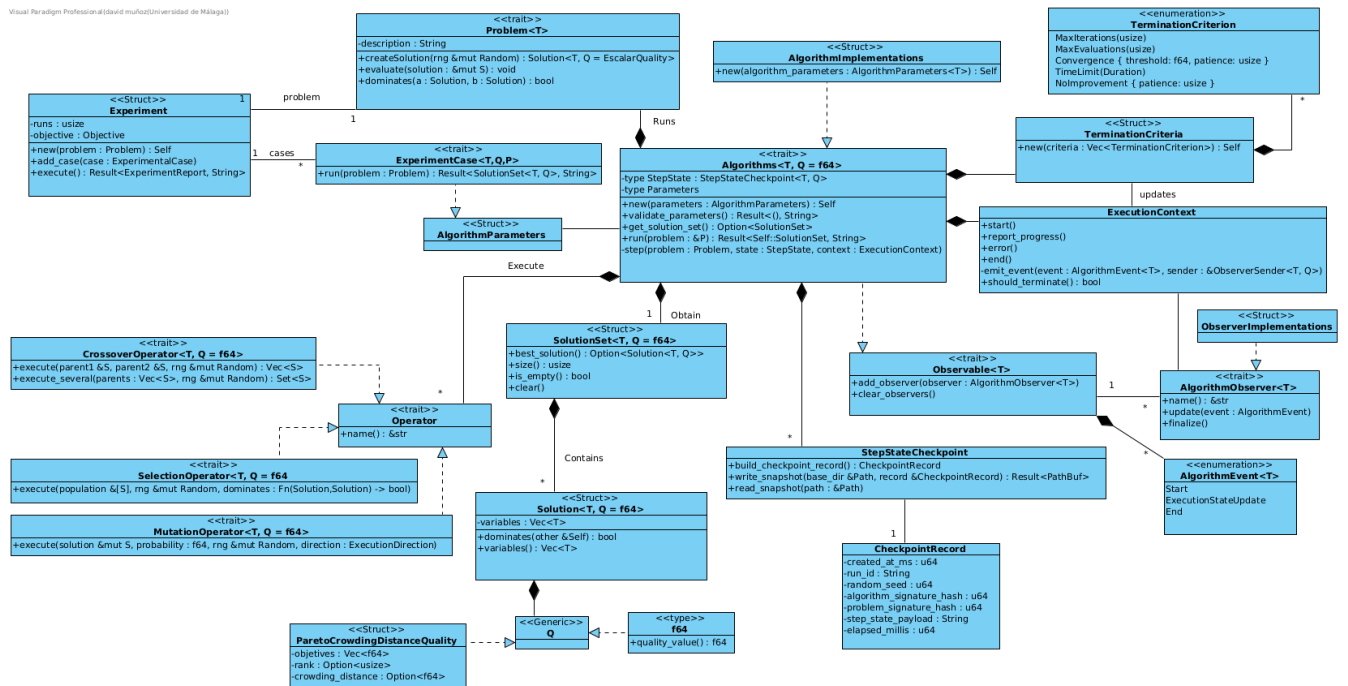


Figura 4.1. Arquitectura de la biblioteca. Diagrama de clases, UML. Visual Paradigm 17.0 [24].

4.2 Módulos

A continuación, se describen los distintos módulos que componen la biblioteca y las responsabilidades asociadas a cada uno de ellos.

4.2.1 Algoritmos

Este módulo es el encargado de ejecutar los distintos problemas definidos en la biblioteca. Este contiene toda la lógica sobre la ejecución, los puntos de control, la comunicación con los observadores y el paralelismo.

Su diseño sigue una separación clara entre las interfaces (traits), la infraestructura de ejecución y

los distintos algoritmos concretos, facilitando la extensibilidad, la reutilización y la mantenibilidad del código.

4.2.1.1 Contenido y organización

- **Interfaces y contratos:** componente que declara las abstracciones y traits que describen el comportamiento esperado de los algoritmos y de sus elementos auxiliares. Estas interfaces permiten sustituir implementaciones concretas sin afectar al resto del sistema.
- **Infraestructura de ejecución:** submódulos encargados de la gestión del ciclo de vida de un experimento/ejecución, incluyendo el *runtime*, la terminación y mecanismos de ejecución asíncrona o en paralelo. Se agrupan aquí las piezas que controlan cuándo y cómo se ejecutan los algoritmos sobre instancias de problema.
- **Persistencia y recuperación:** componentes destinados a checkpointing y mecanismos relacionados que facilitan la recuperación de ejecuciones largas o interrumpidas, y que permiten auditoría y análisis posteriores.
- **Definición de objetivos:** submódulo que centraliza la representación de funciones objetivo y métricas relacionadas, separado de las implementaciones concretas para mantener una clara distinción entre problema y algoritmo.
- **Implementaciones concretas:** paquete separado que contiene las implementaciones específicas de algoritmos (por ejemplo, enfoques evolutivos, heurísticos y de optimización ensamblados). Agruparlas en un submódulo permite añadir, probar y comparar algoritmos sin alterar la infraestructura común.
- **Terminación y políticas:** lógica que alberga criterios de parada y políticas asociadas (por tiempo, por número de iteraciones, por convergencia, etc.), definida aparte para facilitar la composición y reutilización.

4.2.2 Experimentos

El módulo de experimentos constituye la capa de abstracción superior de la biblioteca, diseñada para facilitar la evaluación sistemática y comparativa de los diversos algoritmos frente a un problema específico. Su propósito fundamental es permitir que el usuario final realice un proceso de *benchmarking* robusto, identificando qué configuraciones ofrecen un rendimiento óptimo mediante la ejecución controlada y automatizada de baterías de pruebas.

Este servicio se articula a través de la definición de casos de prueba, los cuales representan combinaciones específicas de parámetros y componentes. Gracias a la arquitectura modular de `roma`, el sistema permite explorar un amplio espacio de configuraciones, incluyendo la variación de operadores, el ajuste de las probabilidades de mutación y cruce, o la selección de diferentes criterios de terminación. Asimismo, es posible comparar directamente algoritmos distintos bajo las mismas condiciones de contorno, lo que proporciona una base empírica sólida para la toma de decisiones técnicas.

Al finalizar las ejecuciones, el módulo genera un reporte detallado que sintetiza los resultados obtenidos por cada configuración. Este informe no solo recopila las soluciones halladas, sino que ofrece una visión analítica del comportamiento de los algoritmos, permitiendo al usuario discernir con criterio qué estrategia es la más adecuada para la resolución de su problema.

4.2.3 Observadores

El compartimento asociado a los observadores se ha diseñado como un componente desacoplado encargado de gestionar la monitorización durante el proceso evolutivo. Esta infraestructura se articula fundamentalmente a través de la definición de eventos específicos, los cuales son emitidos por el motor de ejecución para señalar hitos relevantes o cambios en el progreso del algoritmo. Para dotar de contenido a estos eventos, el sistema emplea estructuras de datos especializadas que encapsulan el estado actual de la ejecución, permitiendo que la información fluya hacia los observadores de manera coherente y estructurada.

4.2.4 Operadores

Este componente constituye el núcleo funcional y arquitectónico de la biblioteca de optimización, ya que encapsula la lógica algorítmica fundamental mediante un modelo desacoplado y altamente modular. En lugar de ligar los operadores a algoritmos específicos, la biblioteca implementa un diseño basado en abstracciones y polimorfismo que separa completamente la definición del espacio de búsqueda de los mecanismos de variación y selección. Este modelo permite que cualquier operador sea tratado como una entidad independiente e intercambiable, facilitando la hibridación de técnicas y permitiendo al usuario configurar algoritmos a la carta combinando distintas estrategias.

Más allá de su flexibilidad arquitectónica, la eficiencia computacional de estos operadores es un factor crítico para el éxito de la biblioteca. Debido a la naturaleza iterativa de las metaheurísticas, en cada ciclo del algoritmo se invocan de manera encadenada múltiples operadores que realizan cálculos y mutaciones directas sobre las estructuras de las soluciones.

Dentro de esta arquitectura flexible y de alto rendimiento, los componentes se clasifican en tres

tipos principales de operadores, diseñados para interactuar perfectamente sobre las estructuras de datos poblacionales:

- **Operadores de mutación:** Introducen variaciones aleatorias en los individuos de la población con el objetivo de mantener la diversidad genética y evitar la convergencia prematura hacia soluciones subóptimas. Estos operadores permiten explorar nuevas regiones del espacio de búsqueda mediante modificaciones controladas de las soluciones existentes.
- **Operadores de selección:** Se encargan de elegir los individuos que participarán en la generación de nuevas soluciones, generalmente en función de su calidad o valor de aptitud. Su función principal es favorecer la propagación de las mejores soluciones encontradas hasta el momento, equilibrando la presión selectiva con la preservación de la diversidad poblacional.
- **Operadores de cruce:** Combinan la información de dos o más individuos seleccionados para generar nuevos descendientes. A través de este proceso se busca explotar las características favorables de las soluciones existentes, recombiniéndolas de manera que puedan dar lugar a individuos con un mejor desempeño.

4.2.5 Problemas

Este módulo constituye la abstracción que define y modela el dominio del escenario de optimización concreto que se desea resolver. Dentro de la arquitectura de la biblioteca, el componente de problemas actúa como un intermediario o contrato formal que desacopla la lógica genérica de las metaheurísticas de los detalles particulares de cada aplicación. Estas estructuras no solo almacenan los datos de entrada o las instancias del problema, sino también las reglas matemáticas y las restricciones que gobiernan el espacio de búsqueda, guiando de manera activa tanto a los algoritmos poblacionales como a los operadores de variación hacia los objetivos definidos. Para cumplir con esta función de guía y control, el módulo asume de forma integrada la responsabilidad de orquestar la creación de soluciones y su respectiva evaluación.

En primer lugar, el módulo define los mecanismos necesarios para construir configuraciones de partida válidas dentro del espacio de búsqueda, soluciones puramente aleatorias, esenciales para garantizar la diversidad en las primeras etapas de las metaheurísticas. Conectado directamente con este proceso, el componente implementa la lógica matemática de la función u objetivos $f(x)$ que miden el desempeño de cada vector de decisión. El módulo es, por tanto, el responsable de transformar una solución abstracta en un valor numérico cualitativo o en un vector de valores si se trabaja en

entornos multiobjetivo, gestionando además el cálculo de las penalizaciones en caso de que la solución generada vulnere alguna de las restricciones de frontera impuestas por el modelo.

Finalmente, al conocer en detalle cómo se manejan y se relacionan internamente los datos del dominio, este componente proporciona la información estructural indispensable para que los operadores echen mano de las características del problema. Esto permite determinar de manera analítica la vecindad de una solución, facilitando que las perturbaciones y las variaciones locales se realicen con un sentido semántico preciso —como el intercambio de aristas en problemas de rutas o la asignación de tareas en problemas de planificación— en lugar de ejecutar modificaciones ciegas que comprometan la eficiencia del proceso de optimización.

4.2.6 Conjunto de soluciones

Los algoritmos devolverán esta estructura de datos, la cual está diseñada como una estructura abstracta orientada a actuar como una entidad contenedora encargada de encapsular de manera unificada el conjunto de resultados obtenidos tras el proceso de búsqueda. En lugar de retornar una colección nativa o un vector de soluciones en crudo, se opta por este diseño de abstracción desacoplado para centralizar y homogeneizar la información derivada de la ejecución del algoritmo ante el problema planteado. Al adoptar la forma de una interfaz o tipo abstracto, se dota al sistema de una alta flexibilidad, permitiendo que esta estructura pueda ser implementada o extendida de cualquier manera en el futuro para solventar necesidades emergentes del desarrollo sin alterar el núcleo de la biblioteca.

Esta naturaleza abstracta permite albergar internamente desde una lista indexada con las soluciones óptimas o aproximadas que han logrado superar los criterios de convergencia impuestos, hasta estructuras más complejas adaptadas a paradigmas específicos. Al aislar esta lógica dentro de un contrato de datos genérico, se facilita la incorporación dinámica de metadatos adicionales de gran valor para la posterior fase de análisis.

4.2.7 Soluciones

Las soluciones representan las estructuras que almacenan el resultado generado por la aplicación iterativa de los operadores dentro de los algoritmos de optimización. Este componente es de vital importancia para la eficiencia global del sistema, dado que constituye la unidad fundamental con la que opera la biblioteca de manera ininterrumpida a lo largo de todo el ciclo de ejecución. El motor de optimización modifica constantemente estas estructuras, gestiona poblaciones masivas de las mismas, evalúa sus atributos en cada iteración y analiza sus valores para guiar la búsqueda. Por consiguiente, cualquier ineficiencia o sobrecarga en su diseño se amplificaría exponencialmente, comprometiendo

críticamente el rendimiento temporal y el consumo de memoria de los algoritmos.

Bajo esta premisa, la estructura se ha diseñado siguiendo una filosofía minimalista que prioriza la ligereza y la simplicidad estructural. Sin embargo, alcanzar este equilibrio ha supuesto un desafío de desarrollo complejo debido a la necesidad de dar soporte simultáneo tanto a problemas de un único objetivo como a enfoques multiobjetivo. Mientras que en entornos mono-objetivo basta con asociar al vector de variables un único valor escalar de aptitud, los entornos multiobjetivo exigen gestionar vectores de funciones objetivo, rangos de dominancia y métricas de densidad como la distancia de apilamiento.

4.2.8 Utilidades

Para posibilitar el desarrollo de este proyecto y cumplir el objetivo de mantener una arquitectura con *cero dependencias externas*, se han desarrollado componentes auxiliares ligeros integrados en el núcleo de la biblioteca. Estos submódulos reproducen, de forma optimizada y autocontenida, las funcionalidades estrictamente necesarias de *crates* estándar del ecosistema Rust, tales como `rand` [6] para la generación estocástica y `plotters` [10] para la representación gráfica. Esta decisión de diseño permite evitar la inclusión de bibliotecas externas sin renunciar a las capacidades necesarias para el correcto funcionamiento del sistema.

5

Implementación y pruebas

Este capítulo detalla el proceso de construcción de la biblioteca `roma`, trasladando los conceptos teóricos y el diseño arquitectónico descritos en los capítulos anteriores a código real en el lenguaje de programación Rust.

5.1 Implementación

Una vez definida y consolidada la arquitectura de la biblioteca, se inició la fase de construcción modular siguiendo un enfoque ascendente. Contrariamente a lo que se podría suponer inicialmente, situando el foco en los algoritmos o en la definición de los problemas, el desarrollo comenzó por el módulo de las **soluciones**. Esta decisión estratégica se fundamenta en que la estructura que define a la solución es, en realidad, el componente más determinante del sistema, actuando como el eje central sobre el que pivotan todos los demás módulos.

5.1.1 Las complejas soluciones

La importancia de este módulo radica en que la solución es la entidad que los operadores transforman para explorar el espacio de búsqueda y la que los observadores monitorizan constantemente para extraer métricas de rendimiento. Del mismo modo, los problemas dependen de esta estructura para realizar tanto la generación inicial de individuos como su posterior evaluación, mientras que los algoritmos actúan como gestores encargados de almacenarlas y orquestar su flujo a lo largo de las iteraciones.

Debido a esta interdependencia, un diseño deficiente o una implementación poco optimizada de

esta estructura central comprometería no solo la **extensibilidad** de la biblioteca —al dificultar la integración de nuevos problemas o tipos de datos— sino también su **rendimiento computacional**. Dado que la solución es el objeto con mayor frecuencia de acceso y modificación, cualquier ineficiencia en su desarrollo se convertiría inevitablemente en un cuello de botella durante ejecuciones intensivas. Por ello, garantizar una base técnica sólida en este módulo fue la prioridad absoluta antes de proceder con la implementación del resto de componentes.

En una primera aproximación del diseño, `Solution` se definió como un *trait* que declaraba un conjunto de funciones fundamentales, tales como la obtención del valor de *fitness*, el acceso a las variables de la solución o la recuperación de su representación interna. Este *trait* debía ser implementado por distintos *structs*, cada uno adaptado a las necesidades específicas de cada problema.

Por ejemplo, si el problema requería representar una solución como una permutación binaria, el *struct* correspondiente contendría como atributos un vector binario junto con el valor de calidad asociado a dicha solución. Sin embargo, se observó que esta arquitectura introducía una elevada fragmentación en las implementaciones y dificultaba la generalización de la biblioteca para soportar un mayor número de problemas y algoritmos.

Por este motivo, se optó por una solución más abstracta y flexible. En el diseño actual [1](#), `Solution` se define como un *struct* genérico parametrizado por dos tipos: `T` y `Q`. El tipo `T` representa el tipo de dato de las variables de decisión almacenadas en la solución, mientras que `Q` representa el tipo asociado a la evaluación de calidad de dicha solución. De forma predeterminada, este último toma el tipo `f64`.

Listing 1 Estructura desarrollada para las Soluciones

```
1  #[derive(Clone, Debug)]
2  pub struct Solution<T, Q = f64> {
3      variables: Vec<T>,
4      value: Option<Q>,
5  }
```

Sobre esta estructura, en lugar de recurrir a implementaciones rígidas o acopladas de manera convencional, se optó por diseñar fábricas de inicialización mediante constructores específicos para entornos mono-objetivo y multiobjetivo capaces de adaptarse a una amplia variedad de problemas. Este enfoque modular permite instanciar estados con una sintaxis fluida y desacoplada del núcleo del algoritmo. Un caso representativo de esta estrategia se ilustra en el Listado [2](#), donde se muestra el código reducido para la construcción de soluciones binarias (cuyas variables se definen mediante valores

booleanos). En este fragmento de código se aprecia cómo el patrón *Builder* facilita la inicialización homogénea de la estructura —ya sea de forma aleatoria estocástica, parametrizada a ceros o a unos— antes de consolidar el objeto de tipo `Solution<bool>` definitivo a través del método de finalización.

Listing 2 Código reducido sobre el constructor de soluciones binarias

```
1 pub struct BinarySolutionBuilder {
2     variables: Vec<bool>,
3     quality: Option<f64>,
4 }
5
6 impl BinarySolutionBuilder {
7     pub fn ones(size: usize) -> Self {
8         Self {
9             variables: vec![true; size],
10            quality: None,
11        }
12    }
13
14    pub fn zeros(size: usize) -> Self {
15        Self {
16            variables: vec![false; size],
17            quality: None,
18        }
19    }
20
21    pub fn random(size: usize, seed: Option<u64>) -> Self {
22        let mut rng = if let Some(seed) = seed {
23            Random::new(seed)
24        } else {
25            Random::new(seed_from_time())
26        };
27        let variables: Vec<bool> = (0..size).map(|_| rng.coin_flip()).collect();
28        Self {
29            variables,
30            quality: None,
31        }
32    }
33
34    pub fn with_quality(mut self, quality: f64) -> Self {
35        self.quality = Some(quality);
36        self
37    }
38
39
40    pub fn build(self) -> Solution<bool> {
41        finalize_scalar_solution(self.variables, self.quality)
42    }
43 }
```

5.1.2 Abstracción de Problemas y Operadores

Posteriormente se abordó la implementación de los **problemas**, definidos mediante un **trait** que actúa como el contrato de interfaz para cualquier modelo de optimización que se desee resolver. Para garantizar que la biblioteca fuera lo suficientemente flexible como para soportar diversos dominios de problemas —desde optimización continua hasta combinatoria—, se diseñó una estructura altamente genérica.

La interfaz que deben satisfacer todos los modelos se ha proyectado bajo un enfoque minimalista y cohesivo. Este contrato obliga a la implementación de métodos específicos destinados a la obtención de metadatos identificativos de la instancia, los cuales sirven como identificadores unívocos en los sistemas de persistencia y puntos de control (*checkpoints*). Asimismo, exige la definición de funciones para la evaluación cuantitativa de las soluciones y un mecanismo de inicialización estocástica que dote a los algoritmos de un punto de partida válido en el espacio de búsqueda. Esta especificación formal se detalla en el Listado 3.

Listing 3 Definición genérica del **trait Problem** para la abstracción de problemas de optimización.

```
1 pub trait Problem<T, Q = f64>
2 where
3     T: Clone,
4     Q: Clone + Default,
5 {
6     fn new() -> Self;
7
8     fn better_fitness_fn(&self) -> fn(f64, f64) -> bool;
9
10    fn evaluate(&self, solution: &mut Solution<T, Q>);
11
12    fn create_solution(&self, _rng: &mut Random) -> Solution<T, Q>;
13
14    fn dominates(&self, solution_a: &Solution<T, Q>, solution_b: &Solution<T, Q>) -> bool;
15
16    fn get_improvement_direction(&self) -> ImprovementDirection;
17
18    fn get_problem_description(&self) -> String;
19
20    fn format_solution(&self, solution: &Solution<T, Q>) -> String;
21 }
```

Como se observa en la definición conceptual expuesta en el Listado 3, el uso de tipos genéricos permite desacoplar por completo la lógica del problema de la representación interna de los datos.

El parámetro `T` representa el tipo de la variable de decisión, mientras que `Q` define el tipo de la evaluación (u objetivo), cuya especialización por defecto se establece como `f64`. Esta abstracción delega en la concreción del usuario la responsabilidad de definir aspectos críticos como la dirección de mejora (`better_fitness_fn` e `get_improvement_direction`), permitiendo configurar de manera flexible los criterios de ordenación e idoneidad de las soluciones.

Asimismo, el `trait` estandariza la forma en que los algoritmos interactúan con el dominio del problema: la función `evaluate` califica la calidad de un espécimen mediante efectos colaterales controlados en su estado, mientras que `create_solution` define el mecanismo de generación inicial parametrizado por un generador de números pseudoaleatorios. Esta arquitectura asegura que las metaheurísticas puedan operar de forma agnóstica al problema concreto, potenciando la extensibilidad y reutilización del sistema. Una vez consolidada esta base, el desarrollo prosiguió con la implementación de los operadores y los algoritmos, encargados de explotar estas definiciones para guiar la búsqueda hacia soluciones óptimas.

En lo que respecta a los operadores, se definió el `trait Operator` como la abstracción base de la jerarquía del módulo de variación. Aunque su estructura se ha mantenido intencionadamente mínima para no sobrecargar el tiempo de cómputo, actúa como una interfaz común que unifica la gestión de metadatos esenciales, tales como el nombre del operador o su firma procedimental. Esto facilita la trazabilidad de las transformaciones genéticas, la auditoría del comportamiento del algoritmo y la posterior estructuración de los reportes estadísticos de resultados.

Listing 4 Interfaz base `Operator` para la identificación de componentes.

```
1 pub trait Operator {  
2     fn name(&self) -> &str;  
3 }
```

A partir de esta interfaz fundamental, se especializa la jerarquía en tres categorías que representan los pilares de las metaheurísticas: los operadores de **mutación**, **cruce** (*crossover*) y **selección**. Esta especialización mediante subtipos de `traits` permite que los algoritmos manipulen las soluciones de forma genérica, independientemente del tipo de dato subyacente (`T`) o de la métrica de evaluación (`Q`), tal y como se detalla en el Listado 5.

Listing 5 Muestra del desarrollo de los diferentes tipos de `traits` sobre los operadores.

```
1 pub trait MutationOperator<T, Q = f64>: Operator
2 where
3     T: Clone,
4     Q: Clone,
5 {
6     fn execute(
7         &self,
8         solution: &mut Solution<T, Q>,
9         probability: f64,
10        bounds: Option<&RealBounds>,
11        rng: &mut Random,
12    );
13 }
14
15 pub trait CrossoverOperator<T, Q = f64>: Operator
16 where
17     T: Clone,
18     Q: Clone,
19 {
20     fn execute(
21         &self,
22         parent1: &Solution<T, Q>,
23         parent2: &Solution<T, Q>,
24         bounds: Option<&RealBounds>,
25         rng: &mut Random,
26     ) -> Vec<Solution<T, Q>>;
27 }
28
29 pub trait SelectionOperator<T, Q = f64>: Operator
30 where
31     T: Clone,
32     Q: Clone,
33 {
34     fn execute<'a>(
35         &self,
36         population: &'a [Solution<T, Q>],
37         rng: &mut Random,
38         dominates: &dyn Fn(&Solution<T, Q>, &Solution<T, Q>) -> bool,
39     ) -> &'a Solution<T, Q>;
40 }
```

Como se puede apreciar en el Listado 5, cada contrato especifica formalmente los requisitos para su cometido. Un aspecto clave en el diseño de las firmas de `MutationOperator` y `CrossoverOperator` es la inclusión opcional de los límites espaciales del problema mediante el parámetro `bounds: Option<&RealBounds>`. Esta adición es de vital importancia para garantizar

la validez matemática y la abstracción de la biblioteca cuando se opera en espacios de búsqueda continuos de números reales. Operadores paradigmáticos de la literatura, tales como el cruce binario simulado (*Simulated Binary Crossover*, SBX) o la mutación polinomial (*Polynomial Mutation*), calculan las nuevas posiciones basándose estrictamente en distribuciones de probabilidad acotadas por las fronteras del espacio factible Ω . Al pasar estos límites de manera genérica y opcional, la biblioteca es capaz de reutilizar el mismo flujo computacional tanto para problemas combinatorios o discretos (donde los límites reales no aplican y se evalúan como `None`) como para optimización continua de alta dimensionalidad.

Por otra parte, destaca la gestión de los tiempos de vida o *lifetimes* ('a) implementada en la firma del método de ejecución de `SelectionOperator`. En lenguajes de programación tradicionales, retornar un elemento elegido de una colección suele implicar o bien la copia íntegra del objeto en memoria o bien el uso de punteros desprotegidos que amenazan la estabilidad del hilo de ejecución. A través del parámetro genérico de ciclo de vida 'a, el compilador de Rust valida estáticamente que la referencia de la solución seleccionada devuelta por el operador posea exactamente la misma persistencia temporal que el vector de la población original (`population: &'a [Solution<T, Q>]`). Esta restricción asegura de forma matemática que no se produzcan punteros colgados (*dangling pointers*) si la población se destruye, eliminando por completo la necesidad de realizar clonaciones pesadas de memoria (*overhead* por asignación en el *heap*) y optimizando drásticamente la velocidad en cada iteración del algoritmo.

5.1.3 Funcionamiento concreto de los algoritmos

Para la configuración e instanciación de cada algoritmo, el sistema se apoya en una estructura de parámetros que actúa como su base configurativa unificada. Esta estructura encapsula toda la información necesaria para la construcción y parametrización de la metaheurística, desempeñando además un papel crucial en el módulo de experimentos. En dicho módulo, se emplea para almacenar y replicar las configuraciones específicas de cada algoritmo que servirán como casos de prueba o vectores de sintonización dentro de los *benchmarks*.

Con el objetivo de garantizar la integridad y robustez del sistema antes de iniciar el cómputo, se incorpora un mecanismo de verificación estricto gobernado por la función `validate_parameters(&self) -> Result<(), String>`. Este método se invoca de manera sistemática y automatizada antes de comenzar el proceso de resolución, validando que los rangos numéricos, probabilidades y dimensiones introducidos por el usuario sean coherentes y correctos. En caso de detectar anomalías, la función

interrumpe la ejecución de forma segura devolviendo un error con un mensaje personalizado, lo que facilita una auditoría precisa y un diagnóstico rápido de la configuración antes de que el motor consuma tiempo de CPU.

Para llevar a cabo su propósito exploratorio, las metaheurísticas interactúan directamente con los operadores de variación y selección, los cuales realizan transformaciones estocásticas y locales sobre las soluciones de forma iterativa. Gracias al acoplamiento débil que ofrece el diseño arquitectónico, estos operadores son completamente intercambiables; esto permite alterar radicalmente el comportamiento dinámico del algoritmo sin modificar un solo bloque de su implementación interna. Bajo este mismo principio de modularidad y diseño agnóstico, el sistema facilita la sustitución o modificación del problema en evaluación de manera directa y flexible. Asimismo, los algoritmos admiten la inyección de observadores para monitorizar el estado interno de la ejecución en tiempo real, un componente que se detalla minuciosamente en su apartado correspondiente.

Desde el punto de vista del rendimiento temporal, estas estructuras se han diseñado para ser ejecutadas de forma concurrente o paralela, minimizando la latencia de procesamiento. La biblioteca ofrece al usuario la flexibilidad de especificar manualmente el tamaño del grupo de hilos (*thread pool*) a instanciar o, alternativamente, delegar dicha gestión al propio entorno. En este último escenario, el nivel de concurrencia óptimo se determina dinámicamente en tiempo de ejecución analizando la topología de la máquina del usuario, utilizando para ello la función de la biblioteca estándar `available_parallelism()`. Toda esta suite de comportamientos y capacidades operativas queda consolidada formalmente bajo el contrato de interfaz especificado por el `trait Algorithm`, detallado en el Listado 6.

Listing 6 Definición del contrato y de los tipos asociados para el componente `Algorithm`.

```
1 pub trait Algorithm<T, Q = f64>
2 where
3     T: Clone + Send + 'static + Display,
4     Q: Clone + Default + Send + 'static + Display,
5 {
6     type SolutionSet: SolutionSet<T, Q>;
7     type Parameters;
8     type StepState: StepStateCheckpoint<T, Q>;
9
10    fn new(parameters: Self::Parameters) -> Self;
11
12    fn algorithm_name(&self) -> &str;
13
14    fn termination_criteria(&self) -> TerminationCriteria;
15
16    fn observers_mut(&mut self) -> &mut Vec<Box<dyn AlgorithmObserver<T, Q>>>;
17
18    fn set_solution_set(&mut self, solution_set: Self::SolutionSet);
19
20    fn run<P>(&mut self, problem: &P) -> Result<Self::SolutionSet, String>;
21
22    fn validate_parameters(&self) -> Result<(), String> {
23        Ok(())
24    }
25
26    fn get_solution_set(&self) -> Option<&Self::SolutionSet>;
27
28    fn step(
29        &self,
30        problem: &(impl Problem<T, Q> + Sync),
31        state: &mut Self::StepState,
32    );
33 }
```

Como se observa en el Listado 6, la declaración del `trait` hace uso de tipos asociados (`type SolutionSet`, `type Parameters` y `type StepState`). Esta decisión de diseño en lugar de recurrir puramente a parámetros genéricos en la cabecera del rasgo resulta fundamental para limpiar las firmas del código, agrupando los tipos dependientes bajo el espacio de nombres del propio algoritmo y garantizando que un algoritmo concreto solo pueda interactuar con su estructura de parámetros y estados compatible.

Asimismo, es imprescindible destacar la rigurosidad de las restricciones de concurrencia (*trait bounds*) impuestas sobre los parámetros de tipos genéricos `T` (variables de decisión) y `Q` (valores

de aptitud), que obligan a la implementación de los marcadores `Send` y `'static`. En el ecosistema Rust, garantizar de forma estática que las estructuras de datos poblacionales puedan transferirse de forma segura entre las fronteras de distintos hilos de ejecución sin riesgo de provocar condiciones de carrera es una exigencia crítica. La presencia de `Send` asegura esta transferencia segura de la propiedad (*ownership*) en entornos multitarea, mientras que el ciclo de vida 'a implícito en `'static` certifica que los datos contenidos no poseen referencias locales con hilos que puedan ser destruidos prematuramente, blindando la integridad de la memoria.

La lógica de ejecución se implementa mediante un sistema basado en pasos. El punto de entrada principal para iniciar cualquier metaheurística es el método `run`, cuya firma procedimental e interfaz de rasgos se detalla en el Listado 7.

Listing 7 Definición de la firma genérica del método principal `run`.

```
1 fn run<P>(&mut self, problem: &P) -> Result<Self::SolutionSet, String>
2 where
3     Self: Sized,
4     Self::SolutionSet: Clone,
5     P: Problem<T, Q> + Sync,
6 {
```

El diseño de la función `run` aprovecha las ventajas de los sistemas de tipos avanzados de Rust. Destaca en su declaración el uso del límite restrictivo `Self: Sized`. En Rust, por defecto, los *traits* pueden ser implementados por tipos cuyo tamaño en tiempo de compilación no sea conocido de forma estática (como los objetos de rasgo o *trait objects*). Al imponer el *bound* `Sized`, se garantiza que el algoritmo concreto posee un tamaño fijo y conocido en tiempo de compilación. Esto es un requisito indispensable para permitir el polimorfismo estático, asegurando que el compilador optimice las llamadas a funciones eliminando el coste de la resolución dinámica en tiempo de ejecución (**dynamic dispatch**). Asimismo, la restricción `Sync` sobre el parámetro genérico del problema (`P`) asegura que la estructura del modelo matemático sea segura para ser accedida concurrentemente desde múltiples hilos de ejecución, habilitando el paralelismo a nivel del motor experimental.

Desde una perspectiva arquitectónica, `run` delega la orquestación y el control de eventos en una función interna especializada de la biblioteca denominada `run_with_observer_runtime`, detallada en el Listado 8. El método `run` actúa bajo un enfoque de programación funcional, encapsulando el comportamiento interactivo de la metaheurística en una *closure* o función anónima que se inyecta a través del argumento `task`. El motor central evalúa de forma transparente esta tarea, aislando los

mecanismos de monitorización del algoritmo en sí.

Listing 8 Implementación del motor de ejecución y el entorno de comunicación para los observadores.

```
1 pub fn run_with_observer_runtime<T, Q, R, F>(  
2     observers: &mut Vec<Box<dyn AlgorithmObserver<T, Q>>>,  
3     criteria: TerminationCriteria,  
4     better_fitness: fn(f64, f64) -> bool,  
5     algorithm_name: impl Into<String>,  
6     task: F,  
7 ) -> R  
8 where  
9     T: Clone + Send + 'static,  
10    Q: Clone + Send + 'static,  
11    F: FnOnce(&ExecutionContext<T, Q>) -> RuntimeExecutionOutput<R>,  
12 {  
13     let algorithm_name = algorithm_name.into();  
14  
15     let runtime = ObserverRuntime::new(std::mem::take(observers));  
16     let context = ExecutionContext::new(runtime.sender(), criteria, better_fitness);  
17     context.start(algorithm_name);  
18  
19     let output = task(&context);  
20     context.end(output.total_generations, output.total_evaluations);  
21  
22     drop(context);  
23  
24     *observers = runtime.finish();  
25     output.result  
26 }
```

Como se aprecia en la lógica de `run_with_observer_runtime`, la función asume el control del ciclo de vida del proceso de optimización. Inicialmente extrae los observadores mediante `std::mem::take` para evitar copias costosas y configura un canal de comunicación asíncrono gestionado a través de un `ExecutionContext`. Tras arrancar formalmente el contexto, se invoca la *closure* `task`, transfiriéndole una referencia del contexto de ejecución para que el algoritmo pueda reportar sus progresos. Una vez finalizada la tarea, es de vital importancia la llamada explícita a `drop(context)`. Este paso destruye el contexto de forma controlada, provocando el cierre definitivo del extremo emisor del canal (**channel**). Al cerrarse el flujo, el hilo secundario encargado de procesar la lógica de los observadores puede consumir los mensajes restantes en cola (**drain**) y concluir de manera limpia su ciclo de ejecución, evitando interbloqueos (**deadlocks**) antes de recuperar los observadores a través de `runtime.finish()`.

Finalmente, la sustancia algorítmica de la tarea inyectada se despliega en un bucle iterativo controlado por las condiciones de parada, cuyo comportamiento se describe en el Listado 9.

Listing 9 Bucle de ejecución del interior de la función `run`

```
1 while !context.should_terminate()
2 {
3     algorithm.step(problem, &mut state);
4
5     let step_snapshot = algorithm.build_snapshot(problem, &state);
6     context.update_execution_state(&step_snapshot);
7     last_iteration = step_snapshot.iteration;
8     last_evaluations = step_snapshot.evaluations;
9
10    if DEFAULT_FREQUENCY_OF_CHECKPOINT_WRITES > 0 &&
11    step_snapshot.iteration % DEFAULT_FREQUENCY_OF_CHECKPOINT_WRITES == 0
12    {
13        let record = state.build_checkpoint_record(
14            &run_id,
15            &checkpoint_metadata,
16            context.time_elapsed(),
17        );
18        checkpoint_policy.persist_record(&mut last_checkpoint_path, &record);
19    }
20    context.report_progress(
21        ObserverState::from_snapshot(&step_snapshot, context.seq_id())
22    );
23 }
```

En cada iteración del bucle, el algoritmo avanza mediante el método `step`, el cual altera el estado de la población o la trayectoria actual. Acto seguido, se extrae un estado intermedio inmutable o captura (*snapshot*) que almacena la evolución temporal de la ejecución. Este reporte intermedio se aprovecha por el sistema de tolerancia a fallos mediante una política de puntos de control o *checkpoints*. Si la frecuencia parametrizada coincide con el módulo de la iteración corriente, se construye un registro (*record*) que se vuelca de forma persistente a disco a través de `checkpoint_policy.persist_record`, permitiendo reanudar la computación en caso de una interrupción imprevista de la máquina. Este sistema es el encargado de guardar la información

El ciclo concluye con la invocación de `context.report_progress`, función encargada de serializar y transmitir el nuevo estado en forma de un mensaje `ObserverState` hacia el canal del runtime de observadores, garantizando una auditoría en tiempo real del proceso sin penalizar el rendimiento del hilo principal.

El bucle de ejecución persiste mientras el contexto del algoritmo, representado por la estructura `ExecutionContext`, no determine el cese de la actividad mediante el método `should_terminate()`. La decisión de terminación está delegada de forma flexible en el tipo `TerminationCriterion`, un enumerado que permite al usuario definir condiciones de parada basadas en diversos criterios físicos y lógicos.

Listing 10 Enumerado `TerminationCriterion` con las condiciones de parada disponibles.

```
1      #[derive(Clone, Debug)]
2      pub enum TerminationCriterion {
3          /// Número máximo de iteraciones (generaciones, pasos, etc.).
4          MaxIterations(usize),
5          /// Número máximo de evaluaciones de la función objetivo.
6          MaxEvaluations(usize),
7          /// Criterio de convergencia basado en un umbral de cambio relativo.
8          Convergence { threshold: f64, patience: usize },
9          /// Límite de tiempo real de ejecución.
10         TimeLimit(Duration),
11         /// Parada por estancamiento (falta de mejora en la calidad).
12         NoImprovement { patience: usize },
13     }
```

Esta implementación mediante un `enum` permite una gran expresividad en la configuración de los experimentos. El usuario puede optar por límites tradicionales, como el número de iteraciones o evaluaciones, o por criterios más avanzados de convergencia y estancamiento (*patience*). La integración de estos criterios en el `ExecutionContext` garantiza que la comprobación se realice de manera eficiente al finalizar cada paso del algoritmo, permitiendo además que los observadores capturen el progreso en tiempo real antes de cada evaluación de terminación.

5.1.4 Observadores y checkpoints

Los observadores se basan en el uso de *traits*, los cuales actúan como interfaces abstractas que definen el contrato de comportamiento obligatorio para cualquier componente de monitorización. Este enfoque no solo garantiza un polimorfismo robusto, sino que facilita la extensibilidad del sistema, permitiendo al desarrollador integrar nuevas estrategias de visualización o registro sin necesidad de alterar el núcleo de la biblioteca. Para cubrir las necesidades de monitorización más comunes, el módulo incorpora diversas implementaciones predefinidas con un funcionamiento por defecto. Estas herramientas listas para su uso inmediato abarcan desde la salida formateada por consola estándar (*stdout*) hasta sistemas encargados de estructurar datos evolutivos en tablas, renderizar gráficos

dinámicos de rendimiento y generar reportes analíticos detallados en formato HTML. Estas dos últimas variantes permiten examinar minuciosamente la convergencia de las metaheurísticas en función del número acumulado de evaluaciones de la función objetivo.

Para integrar este ecosistema, los observadores se asocian de forma dinámica a aquellos algoritmos que satisfacen el contrato del *trait Observable*, interfaz que provee los métodos requeridos para registrar y revocar suscripciones durante el ciclo de vida de la ejecución.

Sin embargo, el diseño del flujo de comunicación entre el motor algorítmico y los observadores adjuntos supuso un reto de diseño de software debido a las restricciones impuestas por el sistema de propiedad y préstamos de Rust (*Borrow Checker*). Inicialmente, se evaluó una implementación clásica basada en el patrón *Observer* convencional, donde el objeto del algoritmo retenía una colección de referencias mutables a sus observadores para notificarlos secuencialmente a través de un método directo. Esta aproximación resultó excesivamente compleja y rígida, ya que las estrictas reglas que prohíben la coexistencia de múltiples alias compartidos con acceso mutable (*aliasing + mutability*) imposibilitaban un manejo limpio de los tiempos de vida en entornos concurrentes.

Como alternativa de diseño, se optó por un modelo concurrente basado en canales (*channels*), consolidando un patrón idiomático que garantiza la seguridad de memoria sin recurrir a mecanismos costosos de exclusión mutua. En este esquema, el algoritmo de optimización se delega a un hilo de ejecución independiente, mientras que la suite de observadores permanece en el hilo principal a la espera de eventos de forma asíncrona mediante el modelo de paso de mensajes. Bajo este paradigma, el hilo del algoritmo actúa como un productor exclusivo que genera capturas inmutables del estado actual (*snapshots*) al completar cada iteración o alcanzar un hito de progreso, enviándolas a través del canal. El hilo principal asume el rol de consumidor, encargándose de recibir dichos paquetes de datos de forma no bloqueante y distribuirlos sincronamente entre los observadores registrados, tal y como se ilustra en la Figura 5.1.

Este diseño arquitectónico no solo incrementa la robustez global del sistema, sino que favorece la escalabilidad horizontal de la biblioteca. La incorporación de canales con capacidad de almacenamiento intermedio faculta a los observadores para absorber y ejecutar de forma diferida tareas computacionalmente pesadas o de alta latencia de entrada/salida —como el volcado persistente en disco— en su propio espacio de hilos. Como consecuencia directa, estas operaciones costosas se procesan sin comprometer en ningún momento la velocidad del motor heurístico interno, maximizando la eficiencia temporal y garantizando una gestión óptima de los recursos multi-núcleo de la CPU.

Complementariamente, el sistema de estados no solo asiste a la monitorización externa, sino que

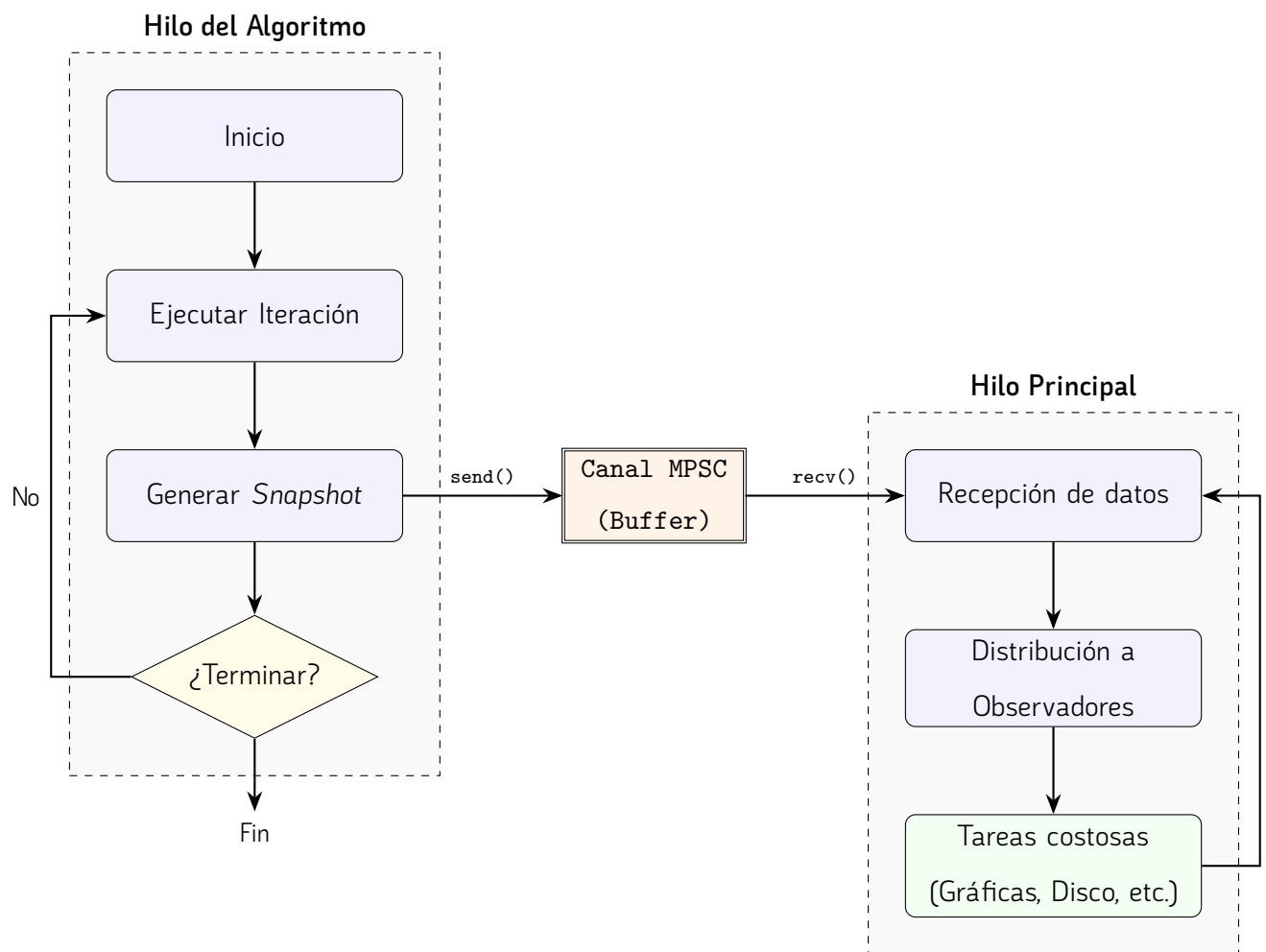


Figura 5.1. Arquitectura de comunicación asíncrona entre hilos mediante eventos.

constituye el pilar fundamental del mecanismo de puntos de control o *checkpoints*. Estos estados se persisten en disco de forma periódica siguiendo las convenciones y rutas de almacenamiento estándar de cada sistema operativo, empleando directorios específicos como `LOCAL_STATE_ROMA` en entornos Linux, `HOME_ROMA` en macOS y `LOCAL_APPDATA` en sistemas Windows.

La información almacenada en cada punto de control incluye los datos críticos necesarios para reconstruir una ejecución específica, abarcando la configuración del algoritmo, el problema abordado y sus parámetros correspondientes. Para garantizar la integridad y la identificación unívoca de cada escenario, este conjunto de datos se procesa mediante una función *hash*, generando un identificador único que vincula de manera determinista la configuración con sus archivos de estado. Al iniciar el sistema con la opción de reanudación activada, la biblioteca genera el *hash* de la configuración actual y busca coincidencias en el sistema de archivos. En caso de localizar puntos de control compatibles, el sistema permite al usuario seleccionar el estado más conveniente para recuperar la ejecución o, alternativamente, optar por iniciar un proceso completamente nuevo, ofreciendo así una capa de tolerancia a fallos y flexibilidad en experimentos de larga duración.

Listing 11 Ejemplo solicitud de punto de control, al intentar reanudar la ejecución de un algoritmo

```

1 david ~/roma/roma/examples cargo run --example knapsack_hc_demo -- --resume
2   Compiling roma v0.1.0 (/home/david/roma/roma)
3   Finished `dev` profile [unoptimized + debuginfo] target(s) in 1.08s
4   Running `/home/david/roma/roma/target/debug/examples/knapsack_hc_demo --resume`
5
6                               --- CHECKPOINT SELECTION ---
7 ID   | AGE      | ELAPSED. | INFO
8 -----
9 [ 1] | 3 days ago | 00:00:00 | > iter=59750;eval=59751;seed=8238046302331804803;
10 [ 2] | 3 days ago | 00:00:00 | > iter=52560;eval=52561;seed=8364640122973076249;
11 [ 3] | 3 days ago | 00:00:00 | > iter=52100;eval=52101;seed=10450884331206695397;
12 [ 4] | 3 days ago | 00:00:00 | > iter=43510;eval=43511;seed=15322852344410324979;
13 [ 5] | 3 days ago | 00:00:00 | > iter=69010;eval=69011;seed=7946725581494022999;
14 [ 6] | 3 days ago | 00:00:00 | > iter=57640;eval=57641;seed=5376056338251316545;
15 [ 7] | 3 days ago | 00:00:00 | > iter=56170;eval=56171;seed=14048048055183702911;
16 -----
17 [0] Start a new run (ignore existing)
18                               -----
19 > Select checkpoint index:

```

5.1.5 Experimentos

——— Implementación

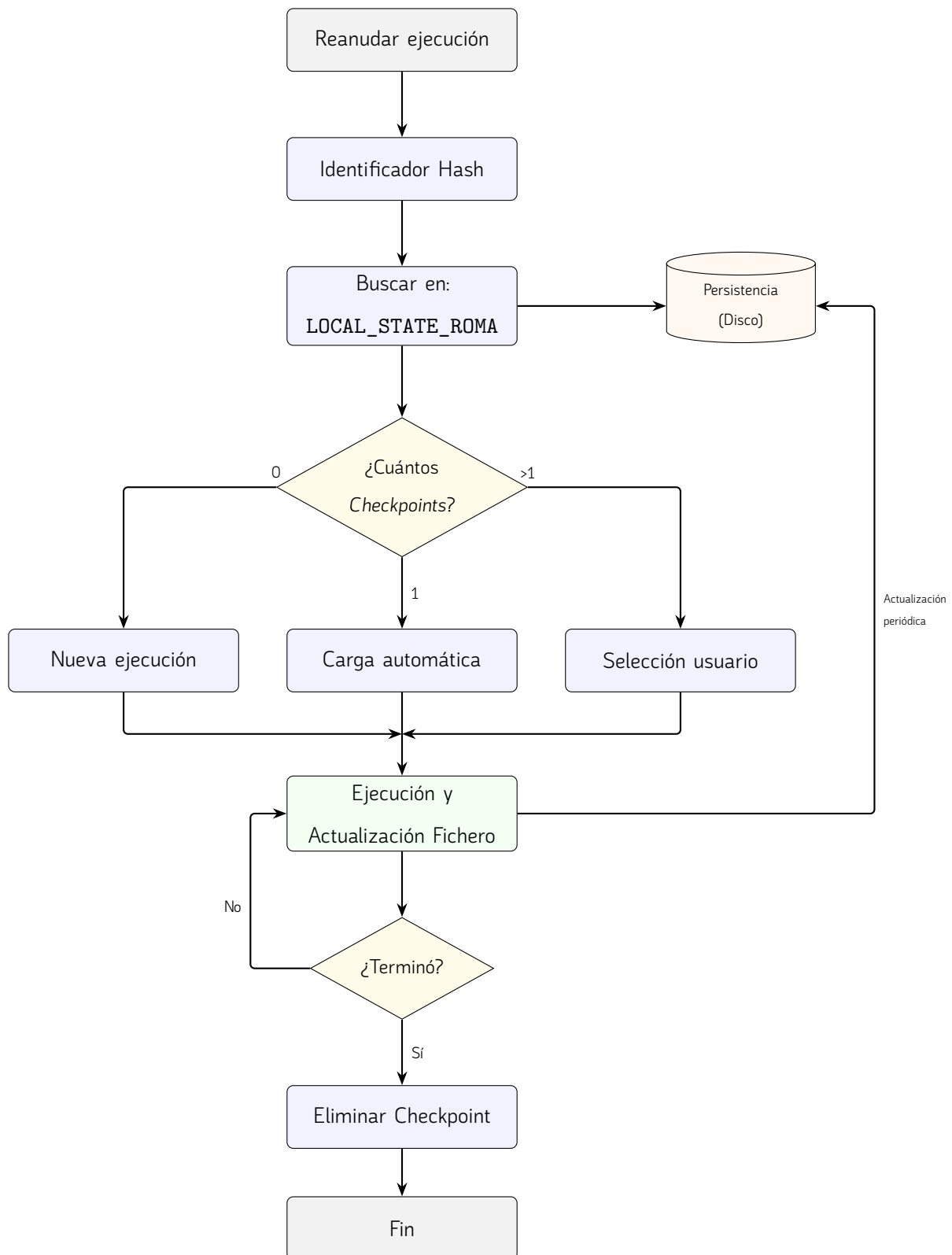


Figura 5.2. Flujo compacto de gestión de persistencia y recuperación de estados en Linux.

Para lograr este grado de flexibilidad y modularidad, el sistema se apoya en abstracciones del lenguaje mediante la definición de *traits* específicos que deben implementar los parámetros de los algoritmos. Estos componentes actúan como interfaces genéricas que habilitan la condición de “experimentable” para cualquier técnica que se integre en la biblioteca, independientemente de sus particularidades internas. Gracias a esta arquitectura, el motor de experimentos puede manipular, instanciar y variar dinámicamente las configuraciones sin necesidad de acoplarse a una implementación concreta.

——— ——— La infraestructura encargada de la ejecución incorpora, asimismo, un sistema de paralelización diseñado para optimizar el uso de los recursos computacionales disponibles. Dado que las metaheurísticas poseen una naturaleza estocástica y requieren ejecuciones independientes recurrentes para obtener significancia estadística, el módulo distribuye de forma eficiente estas tareas concurrentes a nivel de hilos o procesos. Finalmente, el ciclo de experimentación concluye con un subsistema de reporte, el cual recopila las métricas de rendimiento y calidad de las soluciones generadas durante las distintas ejecuciones para exportarlas en formatos estructurados que faciliten su posterior análisis estadístico y visualización.

5.2 Desarrollo de pruebas

La utilización de un correcto conjunto de pruebas agiliza en gran medida el trabajo de programación en cualquier fase del desarrollo. La utilización de un correcto conjunto de pruebas agiliza en gran medida el trabajo de programación en cualquier fase del desarrollo, previniendo regresiones y garantizando la estabilidad del código. Poder verificar de forma determinista que el código subyacente opera de manera correcta resulta indispensable. Para llevar a cabo esta tarea, el proyecto se ha apoyado fuertemente en el *framework* de pruebas integrado de forma nativa en el ecosistema de Rust (`cargo test`).

5.2.1 Pruebas unitarias

Dada la metodología de desarrollo ascendente adoptada, la implementación de pruebas unitarias se convirtió en un pilar fundamental para asegurar la robustez de cada componente antes de escalar hacia niveles de abstracción superiores. El objetivo principal de estas pruebas fue verificar de forma aislada e independiente el comportamiento de las funciones y estructuras mínimas de cada módulo, garantizando que cada unidad de código cumpliera estrictamente con su responsabilidad única.

En consonancia con las prácticas estándar del ecosistema Rust, estas pruebas unitarias se encuentran integradas directamente en el mismo archivo que el código fuente del módulo que evalúan. Generalmente ubicadas al final de cada archivo bajo un módulo específico anotado con el atributo

`#[cfg(test)]`, esta disposición permite mantener una relación estrecha entre la implementación y su verificación. Además, esta estructura facilita el acceso a elementos privados del módulo, permitiendo una validación exhaustiva de la lógica interna que no sería accesible desde el exterior, garantizando así que cada componente sea intrínsecamente correcto antes de ser expuesto.

Este enfoque de aislamiento resultó especialmente crítico en el desarrollo de la estructura de los operadores, donde pequeñas desviaciones en la lógica podrían propagar errores difíciles de detectar en fases posteriores. Al diseñar pruebas simples y focalizadas en una única funcionalidad, se facilitó la identificación inmediata de regresiones durante el proceso de refactorización. Además, estas pruebas permitieron validar la gestión de tipos y la integridad de los datos en tiempo de ejecución, asegurando que los cimientos de la biblioteca fueran sólidos antes de proceder a la integración de sistemas más complejos.

Listing 12 Prueba unitaria que prueba el correcto funcionamiento de la función `dominance` en rangos diferentes para `pareto_crowding_solution.rs`.

```
1      #[test]
2      fn dominates_prefers_lower_rank() {
3          let a = MultiObjectiveRealSolutionBuilder::from_variables(vec![0.0, 0.0])
4              .with_rank(0)
5              .with_crowding_distance(0.1)
6              .build();
7          let b = MultiObjectiveRealSolutionBuilder::from_variables(vec![0.0, 0.0])
8              .with_rank(1)
9              .with_crowding_distance(0.1)
10             .build();
11
12         assert!(a.dominates(&b));
13         assert!(!b.dominates(&a));
14     }
```

5.2.2 Pruebas de integración

Una vez validada la fiabilidad de los componentes individuales mediante el testeo unitario, se procedió al desarrollo de las pruebas de integración. Siguiendo las convenciones de estructuración de proyectos en Rust, este conjunto de pruebas se localiza de forma independiente en el directorio `tests/`, situado en la raíz del proyecto `roma`. Esta separación física respecto al código fuente alojado en `src/` es fundamental, ya que permite tratar a la biblioteca como una entidad externa, evaluando exclusivamente su interfaz pública y la interacción entre sus diversos módulos.

Mientras que las pruebas previas evalúan piezas aisladas, estas se han diseñado con el propósito

de comprobar que todos los componentes de `roma` interactúan armónicamente entre sí para resolver un problema de optimización de principio a fin. El foco de estas pruebas reside en la orquestación del flujo de trabajo: desde la configuración e instanciación de un algoritmo, pasando por la validación sistemática de sus parámetros, hasta la ejecución iterativa donde intervienen operadores, problemas y observadores en un entorno compartido.

Estas pruebas de mayor nivel permiten verificar que la comunicación entre módulos —por ejemplo, el intercambio de soluciones entre el algoritmo y el problema— se realiza de forma eficiente y sin errores de concordancia. Al ejecutarse desde la raíz del proyecto, estas pruebas simulan el uso real que tendría un usuario final de la biblioteca. En última instancia, las pruebas de integración garantizan que el sistema completo es capaz de cumplir con su propósito funcional, ofreciendo una visión global de la estabilidad y el rendimiento de la biblioteca ante casos de uso reales.

Listing 13 Prueba de integración sobre un ejemplo de error donde el tamaño de la población se inicializa a cero.

```
1  #[test]
2  fn ga_returns_error_with_invalid_parameters_edge_case() {
3      // Edge case: invalid parameter configuration should return a validation error.
4      let problem = KnapsackBuilder::new()
5          .with_capacity(10.0)
6          .add_item(5.0, 10.0)
7          .build();
8
9      let parameters = GeneticAlgorithmParameters::new(
10         0,
11         0.8,
12         0.1,
13         SinglePointCrossover::new(),
14         BitFlipMutation::new(),
15         BinaryTournamentSelection::new(),
16         TerminationCriteria::new(vec![TerminationCriterion::MaxIterations(5)]),
17     )
18     .with_seed(1);
19
20     let mut algorithm = GeneticAlgorithm::new(parameters);
21     let error = match algorithm.run(&problem) {
22         Ok(_) => panic!("GA must return an error for invalid parameters"),
23         Err(message) => message,
24     };
25     assert!(error.contains("population_size"));
26 }
```

6

Validación experimental

En este capítulo se presenta la validación práctica de `roma` mediante una serie de experimentos diseñados para evaluar tanto la funcionalidad como la flexibilidad de la biblioteca. Estas pruebas se han integrado en el sistema nativo de ejemplos de `Cargo`, lo que permite que cualquier usuario pueda reproducir los resultados de forma directa. Para ejecutar estos escenarios de prueba, se utiliza el comando estándar del ecosistema Rust: `cargo run --example <nombre_ejemplo>`.

El primero de estos escenarios es el denominado `mono_objective_experiment`. Este experimento tiene como objetivo resolver una instancia del problema de la mochila (*Knapsack Problem*) comparando la eficacia de diversas estrategias algorítmicas bajo condiciones controladas.

Listing 14 Implementación del experimento mono-objetivo (mono_objective_experiment.rs)

utilizando el problema de la mochila.

```
1 use roma::algorithms::{
2     GeneticAlgorithmExperiment, GeneticAlgorithmParameters, HillClimbingParameters,
3     PSOParameters, SimulatedAnnealingParameters, TerminationCriteria, TerminationCriterion,
4 };
5 use roma::experiment::{Experiment, Objective};
6 use roma::operator::{BinaryTournamentSelection, BitFlipMutation, SinglePointCrossover};
7 use roma::problem::KnapsackBuilder;
8
9 fn main() {
10     // Definición de la instancia del problema de la mochila
11     let problem = KnapsackBuilder::new()
12         .with_capacity(150.0)
13         .add_item(1.0, 2.0)
14         .add_item(45.0, 100.0)
15         .add_item(90.0, 301.0)
16         .add_item(150.0, 301.0) // ... (resto de items)
17         .build();
18
19     // Configuración de los casos de prueba (Algoritmos)
20     let hill_climbing_case = HillClimbingParameters::new(
21         BitFlipMutation::new(),
22         0.12,
23         TerminationCriteria::new(vec![TerminationCriterion::MaxIterations(180)]),
24     );
25
26     let genetic_algorithm_case = GeneticAlgorithmExperiment::new(
27         GeneticAlgorithmParameters::new(
28             80, 0.90, 0.06,
29             SinglePointCrossover::new(),
30             BitFlipMutation::new(),
31             BinaryTournamentSelection::new(),
32             TerminationCriteria::new(vec![TerminationCriterion::MaxIterations(60)]),
33         )
34         .with_elite_size(1)
35         .with_threads(4),
36     );
37
38     let simulated_annealing_case = SimulatedAnnealingParameters::new(
39         BitFlipMutation::new(), 0.10, 45.0, 0.985,
40         TerminationCriteria::new(vec![TerminationCriterion::MaxIterations(220)]),
41     ).with_seed(777);
42
43     let pso_case = PSOParameters::new(
44         50, 0.72, 1.49, 1.49,
45         TerminationCriteria::new(vec![TerminationCriterion::MaxIterations(120)]),
46     ).with_velocity_clamp(4.0).with_seed(999);
47
48     // Orquestación y ejecución del experimento
49     let report = Experiment::new(problem)
50         .with_runs(24)
51         .with_objective(Objective::Maximize)
52         .add_case(hill_climbing_case)
53         .add_case(genetic_algorithm_case)
54         .add_case(simulated_annealing_case)
55         .add_case(pso_case)
56         .execute();
57
58     match report {
59         Ok(report) => println!("{}", report.to_text_table()),
60         Err(error) => eprintln!("Experiment execution failed: {}", error),
61     }
62 }
```

Como se observa en la implementación, el experimento comienza con la construcción de una instancia del problema mediante `KnapsackBuilder`, definiendo una capacidad máxima y un conjunto de ítems con sus respectivos pesos y valores. Posteriormente, se definen cuatro configuraciones algorítmicas distintas: *Hill Climbing*, Algoritmo Genético, *Simulated Annealing* y Optimización por Enjambre de Partículas (PSO). Cada uno de estos casos permite un ajuste fino de sus parámetros específicos, como el tamaño de la población, las probabilidades de mutación o el uso de semillas (*seeds*) para garantizar la reproducibilidad de los resultados.

Resulta relevante destacar la capacidad de personalización que ofrece la biblioteca; por ejemplo, en el Algoritmo Genético se especifica el uso de paralelismo mediante la creación de 4 hilos de ejecución. Una vez configurados, todos los casos se añaden a una instancia de `Experiment`, la cual se encarga de orquestar 24 ejecuciones por cada algoritmo para asegurar significancia estadística. Finalmente, el sistema genera un reporte detallado en formato de tabla, permitiendo comparar de forma directa el rendimiento y la convergencia de cada estrategia frente al objetivo de maximización propuesto.

7

Evaluación Comparativa y Análisis de Rendimiento

En este capítulo se describe el proceso de validación experimental y análisis comparativo de la biblioteca desarrollada frente a los marcos de trabajo líderes en la literatura: *jMetal*, *jMetalPy*, *pagmo* y *mealpy*. El objetivo es situar nuestra propuesta en términos de eficiencia computacional, capacidad de convergencia y facilidad de integración.

7.1 Bibliotecas de Referencia

Con el fin de situar la biblioteca desarrollada en el ecosistema actual de la computación evolutiva y la optimización metaheurística, se ha realizado una selección de herramientas de referencia. Estas bibliotecas representan diversos paradigmas de programación, desde implementaciones académicas estrictas hasta plataformas diseñadas para la computación masivamente paralela.

7.1.1 jMetal y jMetalPy

El ecosistema *jMetal* [11] es uno de los marcos de trabajo con mayor trayectoria en la investigación de optimización multiobjetivo. Originalmente desarrollado en Java, destaca por su diseño orientado a objetos basado en interfaces, lo que facilita la extensibilidad de operadores genéticos. Su variante en Python, *jMetalPy* [7], adapta esta arquitectura al entorno científico moderno, integrándose con librerías como *NumPy* y permitiendo la visualización interactiva de frentes de Pareto, lo que la convierte en el estándar para el prototipado académico en entornos de *Data Science*.

7.1.2 Pagmo (C++)

Desarrollada por la Agencia Espacial Europea (ESA), *pagmo* [3] es una plataforma de optimización global masivamente paralela escrita íntegramente en C++. En este estudio se emplea su versión nativa en C++ para evaluar el límite de rendimiento computacional.

7.1.3 DEAP (Distributed Evolutionary Algorithms in Python)

La biblioteca *DEAP* [8] propone un paradigma diferente basado en la metaprogramación. A diferencia de otras herramientas con estructuras rígidas, *DEAP* utiliza un sistema de *toolbox* que permite al usuario construir tipos de datos y algoritmos a medida. Es especialmente reconocida por su capacidad para implementar Programación Genética y por su facilidad para distribuir la evaluación de individuos en clústeres mediante mecanismos de paralelismo simple.

7.1.4 Mealpy

Mealpy [25] es una biblioteca de reciente aparición que se ha posicionado rápidamente debido a su extensísimo catálogo de algoritmos. Se especializa en metaheurísticas bio-inspiradas (como *Swarms*, física o química), ofreciendo una interfaz unificada que prioriza la usabilidad.

7.2 Evaluación con la Función de Rastrigin

Para evaluar el desempeño de la biblioteca en un entorno de búsqueda complejo, se ha seleccionado la función de *Rastrigin*. Este problema es un *benchmark* clásico en optimización matemática debido a su topología altamente multimodal. Basada en la función de esfera pero modificada con un término cosenoidal, la función de *Rastrigin* genera un paisaje de búsqueda plagado de mínimos locales distribuidos regularmente.

Su interés radica en la dificultad que supone para algoritmos de búsqueda: mientras que el óptimo global se encuentra en el origen ($f(\mathbf{x}) = 0$), un algoritmo puede quedar fácilmente atrapado en uno

de los miles de mínimos locales si no gestiona adecuadamente la intensidad de búsqueda. En esta experimentación, se ha configurado una dimensión exigente de $D = 80$ y un presupuesto limitado de 5000 evaluaciones, utilizando el algoritmo *Hill Climbing* para observar cómo cada implementación gestiona la búsqueda en un tiempo crítico bajo condiciones de aislamiento en *Docker*.

7.2.1 Resultados y Análisis

Los resultados consolidados tras 5 ejecuciones independientes con semillas aleatorias fijas se detallan en la Tabla 7.1. Los tiempos reportados corresponden al tiempo medio de ejecución de la CPU dentro del contenedor.

Tabla 7.1. Resultados comparativos: Función de Rastrigin ($D = 80$, 5000 evals).

Biblioteca	Mejor Resultado	Media	Std. Dev.	Tiempo (ms)
Roma (Propuesta)	629.32	664.41	35.36	13.21
jMetal (Java) [11]	528.25	666.17	89.56	83.86
jMetalPy [7]	629.72	695.49	37.30	487.16
pagmo2 (C++) [3]	811.88	894.47	67.93	6.60
DEAP [8]	807.70	861.97	30.62	405.84
Mealpy [25]	859.08	934.23	51.58	373.66

Figura 7.1. Comparativa de convergencia y tiempos de ejecución para $D = 80$.

7.2.2 Discusión de resultados

El análisis de los datos permite extraer conclusiones significativas sobre la competitividad de la biblioteca desarrollada frente a las soluciones establecidas:

- **Precisión y Convergencia:** La propuesta (*Roma*) ha obtenido una media de fitness de 664.41, superando ligeramente a la versión de referencia de *jMetal* en Java (666.17) y siendo significativamente más precisa que las bibliotecas basadas en Python como *DEAP* (861.97) o *Mealpy* (934.23). Es destacable que, aunque *jMetal* Java logró el mejor valor absoluto en una ejecución puntual, nuestra biblioteca presenta una desviación típica mucho menor (35.36 vs 89.56), sugiriendo una mayor robustez.

- **Eficiencia Computacional:** En términos de velocidad, *pagmo2* (C++) lidera la comparativa debido a su naturaleza nativa. Sin embargo, nuestra biblioteca se posiciona como la segunda opción más rápida con 13.21 ms, siendo 6 veces más rápida que *jMetal* Java y hasta 36 veces más veloz que *jMetalPy*.
- **Comportamiento en Alta Dimensionalidad:** Para una dimensión de $D = 80$, muchas implementaciones de Python muestran una degradación en el rendimiento temporal. La biblioteca desarrollada mantiene un consumo de recursos mínimo, lo que valida su arquitectura para problemas de mayor escala.

7.3 Evaluación con el Problema del Viajante (TSP)

Para evaluar el desempeño en problemas de optimización combinatoria, se ha seleccionado el Problema del Viajante (*Traveling Salesman Problem*, TSP). El TSP es un problema NP-duro fundamental en la investigación operativa, cuyo objetivo es encontrar la ruta más corta que visite un conjunto de ciudades exactamente una vez y regrese al punto de partida. En este experimento se ha utilizado la instancia `tsp_48_clustered_euc2d` (48 ciudades) bajo un presupuesto de 4000 evaluaciones.

A diferencia del experimento anterior, aquí se emplea un **Algoritmo Genético (GA)**, lo que permite evaluar la eficiencia de la biblioteca en el manejo de poblaciones, operadores de cruce específicos para permutaciones y gestión de memoria en tareas iterativas de mayor complejidad estructural.

7.3.1 Resultados y Análisis

Los resultados obtenidos tras la orquestación en contenedores se presentan en la Tabla 7.2. Se reporta el fitness (distancia total) y el tiempo de ejecución promedio.

Tabla 7.2. Resultados comparativos: TSP con Algoritmo Genético ($D = 48$, 4000 evals).

Biblioteca	Mejor Fitness	Media Fitness	Std. Dev.	Tiempo (ms)
Roma (Propuesta)	2891.00	3071.40	111.55	5.49
<i>pagmo2</i> (C++) [3]	2578.00	2772.20	166.48	7.55
DEAP [8]	2052.00	2287.60	171.62	154.67
<i>jMetal</i> (Java) [11]	2708.00	3113.20	297.91	47.95
<i>jMetalPy</i> [7]	3414.00	3651.80	169.91	265.48
Mealpy [25]	5117.00	5297.20	100.21	14.39

7.3.2 Interpretación de los resultados

El análisis de la comparativa en el problema TSP revela comportamientos divergentes según la prioridad de la biblioteca:

- **Eficiencia Temporal Extrema:** La biblioteca propuesta, *Roma*, registra el tiempo de ejecución más bajo de toda la comparativa (5.49 ms), superando incluso a *pagmo2* en C++ (7.55 ms) para esta configuración. Esto demuestra una sobrecarga (*overhead*) de gestión de población prácticamente nula, lo cual es crítico cuando se integran estos algoritmos en sistemas de tiempo real.
- **Calidad de la Solución (Fitness):** En este escenario, *DEAP* destaca con la mejor media de fitness (2287.60), lo que sugiere que su implementación interna del Algoritmo Genético para permutaciones está altamente optimizada para la convergencia. *Roma* se sitúa en un rango competitivo, logrando resultados similares a *jMetal* Java (3071 vs 3113) y superando con claridad a *jMetalPy* y *Mealpy*.
- **Robustez:** A pesar de no obtener el fitness más bajo, *Roma* presenta una de las desviaciones típicas más controladas (111.55), lo que indica una alta consistencia entre ejecuciones estocásticas, un factor clave para la fiabilidad de la herramienta.

En conclusión, los resultados del TSP confirman que la biblioteca desarrollada mantiene su ventaja competitiva en términos de velocidad de computación pura, posicionándose como una alternativa eficiente para aplicaciones donde el tiempo de respuesta es tan crítico como la calidad de la solución.

8

Conclusiones y líneas futuras

——— Problemas ———

No determino entre máquinas distintas en ejecuciones paralelas, no se si es un problema, pero si una máquina tiene un procesador con más núcleos que otra máquina, al ejecutar paralelamente un algoritmo con la misma semilla, puede dar lugar a ejecuciones distintas debido a que el algoritmo generará más hilos concurrentes por los cuales puede llegar a conclusiones distintas. (Creo que es importante recalcarlo) —

El desarrollo de este software ha representado un cambio de paradigma respecto a mi formación académica previa. La complejidad técnica no radica meramente en la sintaxis, sino en la comprensión profunda del ecosistema del lenguaje, así como de sus virtudes y limitaciones intrínsecas.

Rust supuso un reto, un lenguaje de sistemas que, si bien permite ciertos patrones de diseño, se aleja de la Programación Orientada a Objetos convencional de lenguajes como Java a la cual estaba acostumbrado. En su lugar, propone un modelo basado en estructuras (structs) y rasgos (traits), estos últimos con una clara influencia de las clases de tipos de Haskell.

Estas variaciones desembocaron en problemas durante el desarrollo de la arquitectura, al no existir tampoco herencia, un factor limitante en el desarrollo de la arquitectura

En cuanto a las herramientas de desarrollo, se utilizó **Visual Studio Code** como entorno de desarrollo integrado (IDE) debido a su flexibilidad, amplia gama de extensiones y soporte para Rust a través de la extensión *rust-analyzer* [2]. Durante el desarrollo más temprano, también se usó **Rust Rover**, de **JetBrains**,

Apéndice A. Installation

Manual

En esta sección se describe el proceso de instalación y puesta en marcha de la biblioteca de optimización metaheurística desarrollada en este trabajo. Dado que el proyecto ha sido implementado íntegramente en Rust, su integración en otros proyectos resulta sencilla y se ajusta a los mecanismos estándar proporcionados por el propio ecosistema del lenguaje.

A.1 Requisitos

Para poder utilizar la biblioteca es necesario disponer de los siguientes elementos:

- Un sistema operativo compatible con el compilador de Rust (Linux, macOS o Windows).
- El compilador de Rust y su gestor de paquetes **Cargo**, instalados y correctamente configurados.
- Acceso a Internet para la descarga del código fuente o del *crate* correspondiente.

La instalación del entorno de desarrollo de Rust puede realizarse siguiendo las instrucciones oficiales proporcionadas por el lenguaje.

A.2 Instalación desde el repositorio

El método más directo para utilizar la biblioteca consiste en clonar el repositorio desde la plataforma GitHub y compilar el proyecto de forma local. Este enfoque resulta especialmente útil para desarrolladores que deseen inspeccionar, modificar o extender el código fuente.

El repositorio puede descargarse ejecutando el siguiente comando:

```
git clone https://github.com/DRLKs/roma
```

Una vez clonado el repositorio, basta con acceder al directorio del proyecto y ejecutar:

```
cargo build
```

Este comando descargará automáticamente las dependencias necesarias y generará la biblioteca en modo desarrollo. Para ejecutar ejemplos incluidos en el repositorio o realizar pruebas, puede utilizarse el comando:

```
cargo run
```

A.3 Instalación como *crate* mediante Cargo

Dado que la biblioteca ha sido diseñada siguiendo las convenciones estándar del ecosistema Rust, también puede integrarse fácilmente como dependencia en cualquier proyecto Rust mediante el gestor de paquetes **Cargo**. Este método es el más recomendado para usuarios que deseen emplear la biblioteca como componente dentro de una aplicación mayor.

Para ello, basta con añadir la dependencia correspondiente en el archivo **Cargo.toml** del proyecto:

```
[dependencies]
roma_lib = "0.1.0"
```

Una vez añadida la dependencia, Cargo se encargará automáticamente de descargar, compilar e integrar la biblioteca durante el proceso de construcción del proyecto. A partir de ese momento, la funcionalidad proporcionada por la biblioteca podrá utilizarse directamente desde el código fuente mediante las instrucciones `use crate::` habituales del lenguaje.

Este mecanismo de distribución facilita la reutilización del software desarrollado y permite mantener actualizada la biblioteca de forma sencilla a medida que se publiquen nuevas versiones.

Bibliografía

- [1] Atlassian. *JIRA: Issue and Project Tracking Software*. Version 9.0. Herramienta para gestión de proyectos, seguimiento de incidencias y colaboración en equipo. URL: <https://www.atlassian.com/software/jira> (visited on 02/17/2026).
- [2] The rust-analyzer authors. *rust-analyzer*. URL: <https://github.com/rust-lang/rust-analyzer> (visited on 02/09/2026).
- [3] Francesco Biscani and Dario Izzo. *pagmo: A C++ platform for massively parallel optimization*. URL: <https://esa.github.io/pagmo2/> (visited on 05/05/2026).
- [4] Christian Blum and Andrea Roli. "Metaheuristics in Combinatorial Optimization: Overview and Conceptual Comparison". In: *ACM Computing Surveys (CSUR)* 35.3 (2003). Artículo de revisión fundamental que introduce, clasifica y compara conceptualmente las principales estrategias metaheurísticas aplicadas a la resolución de problemas de optimización combinatoria, pp. 268–308. DOI: 10.1145/937503.937505. URL: <https://dl.acm.org/doi/10.1145/937503.937505> (visited on 05/16/2026).
- [5] Kalyanmoy Deb et al. "A fast and elitist multiobjective genetic algorithm: NSGA-II". In: *IEEE transactions on evolutionary computation* 6.2 (2002), pp. 182–197.
- [6] The Rand Project Developers and The Rust Project Developers. *rand: Rust random number generators and utilities*. Version 0.10.0. Crate for random number generation in Rust (MIT OR Apache-2.0). URL: <https://github.com/rust-random/rand> (visited on 02/17/2026).
- [7] J. J. Durillo and A. J. Nebro. *jMetalPy*. URL: <https://github.com/jMetal/jMetalPy> (visited on 02/09/2026).
- [8] Félix-Antoine Fortin and colleagues. *DEAP: Distributed Evolutionary Algorithms in Python*. URL: <https://deap.readthedocs.io/en/master/> (visited on 05/05/2026).
- [9] Inc. GitHub. *GitHub: Plataforma de alojamiento de repositorios*. Version N/A. Plataforma de desarrollo colaborativo que permite alojar repositorios Git y facilita la gestión de proyectos de software. URL: <https://github.com/> (visited on 02/17/2026).

- [10] Hao Hou, Aaron Erhardt, and contributors. *Plotters: A Rust drawing and plotting library*. Version 0.3.7. Librería Rust para generar gráficos y visualizaciones de datos. URL: <https://github.com/plotters-rs/plotters> (visited on 02/17/2026).
- [11] A. J. Nebro J. J. Durillo. *jMetal*. URL: <https://github.com/jMetal/jMetal> (visited on 02/09/2026).
- [12] JetBrains s.r.o. *JetBrains Toolbox*. Version 2.3. Aplicación de gestión del ecosistema y suite de entornos de desarrollo integrado (IDEs) profesionales para desarrollo de software y gestión de proyectos. URL: <https://www.jetbrains.com> (visited on 05/16/2026).
- [13] JetBrains s.r.o. *RustRover*. Version 2026.1.1. Entorno de desarrollo integrado (IDE) dedicado para el lenguaje de programación Rust, con soporte nativo para Cargo, asistencia avanzada en la edición de código y herramientas de depuración. URL: <https://www.jetbrains.com/rust/> (visited on 05/16/2026).
- [14] Richard M Karp. "Reducibility among combinatorial problems". In: *Complexity of computer computations*. Springer, 1972, pp. 85–103.
- [15] Thorsten Leemhuis. *Linux Kernel: Rust Support Officially Approved*. heise online. Dec. 2025. URL: <https://www.heise.de/en/news/Linux-Kernel-Rust-Support-Officially-Approved-11109808.html> (visited on 02/15/2026).
- [16] Massachusetts Institute of Technology. *The MIT License*. <https://opensource.org/licenses/MIT>. Accedido el 21 de mayo de 2026. 1988.
- [17] Microsoft Corporation. *Visual Studio Code*. Version 1.104. Editor de código fuente ligero, potente y extensible con soporte integrado para desarrollo de software, depuración y control de versiones. URL: <https://code.visualstudio.com> (visited on 05/16/2026).
- [18] Stack Overflow. *Stack Overflow Developer Survey 2025*. URL: <https://survey.stackoverflow.co/2025/> (visited on 10/12/2025).
- [19] El-Ghazali Talbi. *Metaheuristics: from design to implementation*. John Wiley & Sons, 2009.
- [20] The LaTeX Project. *LaTeX*. Version 2e. Sistema de preparación de documentos para la composición tipográfica de alta calidad, ampliamente utilizado en el ámbito científico y académico. URL: <https://www.latex-project.org> (visited on 05/16/2026).

- [21] The Rust Project Developers. *The Rust Programming Language*. Version 1.95.0. Lenguaje de programación multiparadigma, de código abierto, enfocado en el rendimiento y la seguridad, diseñado para garantizar la gestión segura de la memoria sin necesidad de un recolector de basura. URL: <https://www.rust-lang.org> (visited on 05/16/2026).
- [22] Linus Torvalds and Junio C Hamano. *Git: Distributed Version Control System*. Version 2.44. Sistema de control de versiones distribuido utilizado para el seguimiento de cambios en el código fuente durante el desarrollo de software. URL: <https://git-scm.com/> (visited on 02/17/2026).
- [23] David Muñoz del Valle. *PokerHelper*. URL: <https://github.com/DRLKs/PokerHelper> (visited on 02/09/2026).
- [24] Visual Paradigm International. *Visual Paradigm Professional*. Version 17.0. Herramienta profesional para modelado UML, diseño de arquitectura software y gestión del ciclo de vida del desarrollo. URL: <https://www.visual-paradigm.com> (visited on 02/17/2026).
- [25] Xin-She Yang and colleagues. *MEALPY: MEta-heuristic ALgorithms in PYthon*. URL: <https://mealpy.readthedocs.io/en/latest/pages/general/introduction.html> (visited on 02/09/2026).



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

ETS. de Ingeniería Informática

Bulevar Louis Pasteur, 35

Campus de Teatinos

29071 Málaga